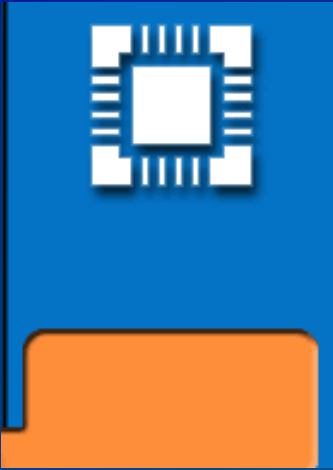
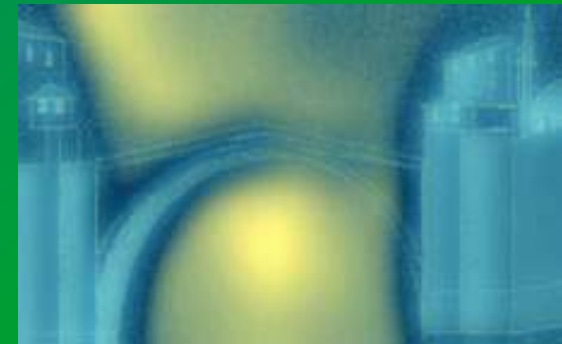




Studij informatike

Objektno orijentirano programiranje



Literatura

- Grady Booch: Object-oriented Analysis and Design With Applications, second edition, Addison-Wesley, Menlo Park, CA, 1994.
- Bjarne Stroustrup: The C++ Programming Language, third edition, Addison-Wesley, Reading, MA, 1997.
- Boris Motik i Julijan Šribar: Demistificirani C++, Element, Zagreb, 1997.
- *Vlastite bilješke*

Praktični rad

- C++
- Visual Basic

Plan predavanja

- Složenost softvera
- Objektni model
- klase i objekti
- Notacija
- klase u Visual Basicu
- COM i DCOM
- Detalji OOP u C++
- CORBA

Složenost programske podrške

- Razlozi složenosti programske podrške
 - složenost i često nejasna definicija problema
 - iterativni razvoj, interakcija s korisnikom
 - teškoće u upravljanju razvojnim procesom
 - dekompozicija u module, koordinacija
 - softver omogućava ostvarenje svih elementarnih gradivnih elementa što se često koristi zbog nedostatka standarda
 - problem opisa ponašanja diskretnih sistema
 - ne mogu se modelirati kontinuiranom funkcijom,
 - mogu imati ogroman broj diskretnih stanja
 - nerazumni zahtjevi korisnika

- Pet atributa složenog sistema
 - imaju hijerarhijsku formu
 - sastoje se od povezanih podsistema koji mogu imati svoje podsisteme
 - definicija osnovnih komponenti nije jednoznačna
 - ovisi o subjektivnom gledištu promatrača
 - veze unutar komponenti su jače od veza među komponentama
 - sistemi su razloživi jer se mogu razdijeliti na komponente
 - sistemi su skoro razloživi jer jer komponente nisu nezavisne
 - često se sastoje od samo nekoliko različitih podsistema, ali u mnogim kombinacijama i pojavnim oblicima
 - osnovne komponente s dna hijerarhije često su zajedničke
 - složeni sistemi evoluiraju iz jednostavnijih
- Razumijevanje složenih sistema olakšava prepoznavanje svojstava koja su zajednička s drugim sistemima
 - avion, automobil

Prevladavanje složenosti programske podrške

■ Dekompozicija

divide et impera (podijeli pa vladaj)

● algoritamska dekompozicija

- problem razlaže na slijed događaja ili funkcija

● objektno-orijentirana dekompozicija

- definira povezanost i međusobnu komunikaciju među objektima u sistemu

■ Apstrakcija

■ Hijerarhija

■ Metode analize i oblikovanja programske podrške

Metoda: proces izrade skupa modela koji opisuju elemente i ponašanje sistema, uz korištenje dobro definirane notacije

Metodologija: skup metoda koje se primjenjuju tokom razvoja programske podrške, a koje posjeduju zajednički idejni pristup

● oblikovanje odozgo prema dolje (*top-down design*)

- prilagođeno strukturiranim programskim jezicima (glavni program, funkcije)
- zasad je još najveći dio postojeće programske podrške oblikovan uz pomoć ovih metoda

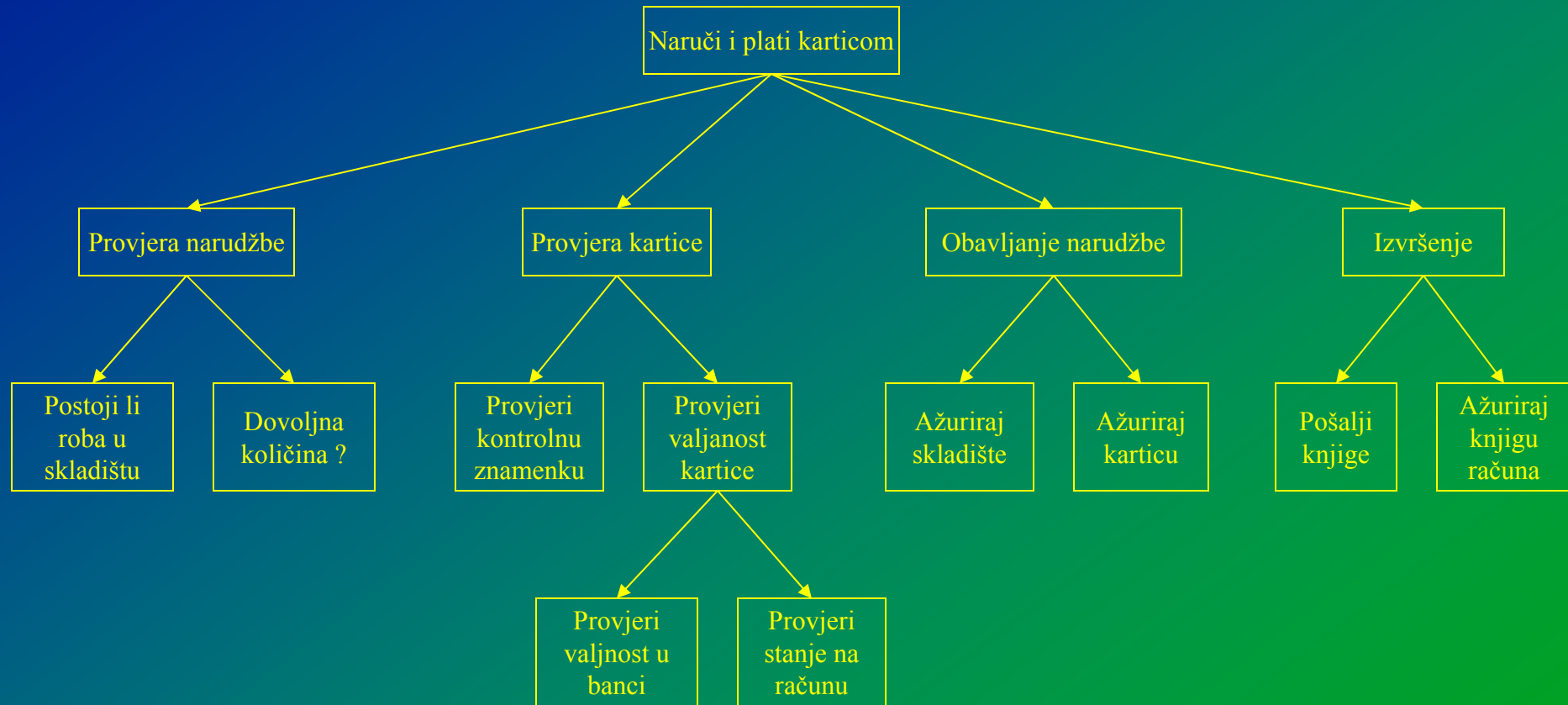
● podacima orijentirano oblikovanje (*data-driven design*)

- preslikava ulaz sistema na izlaz

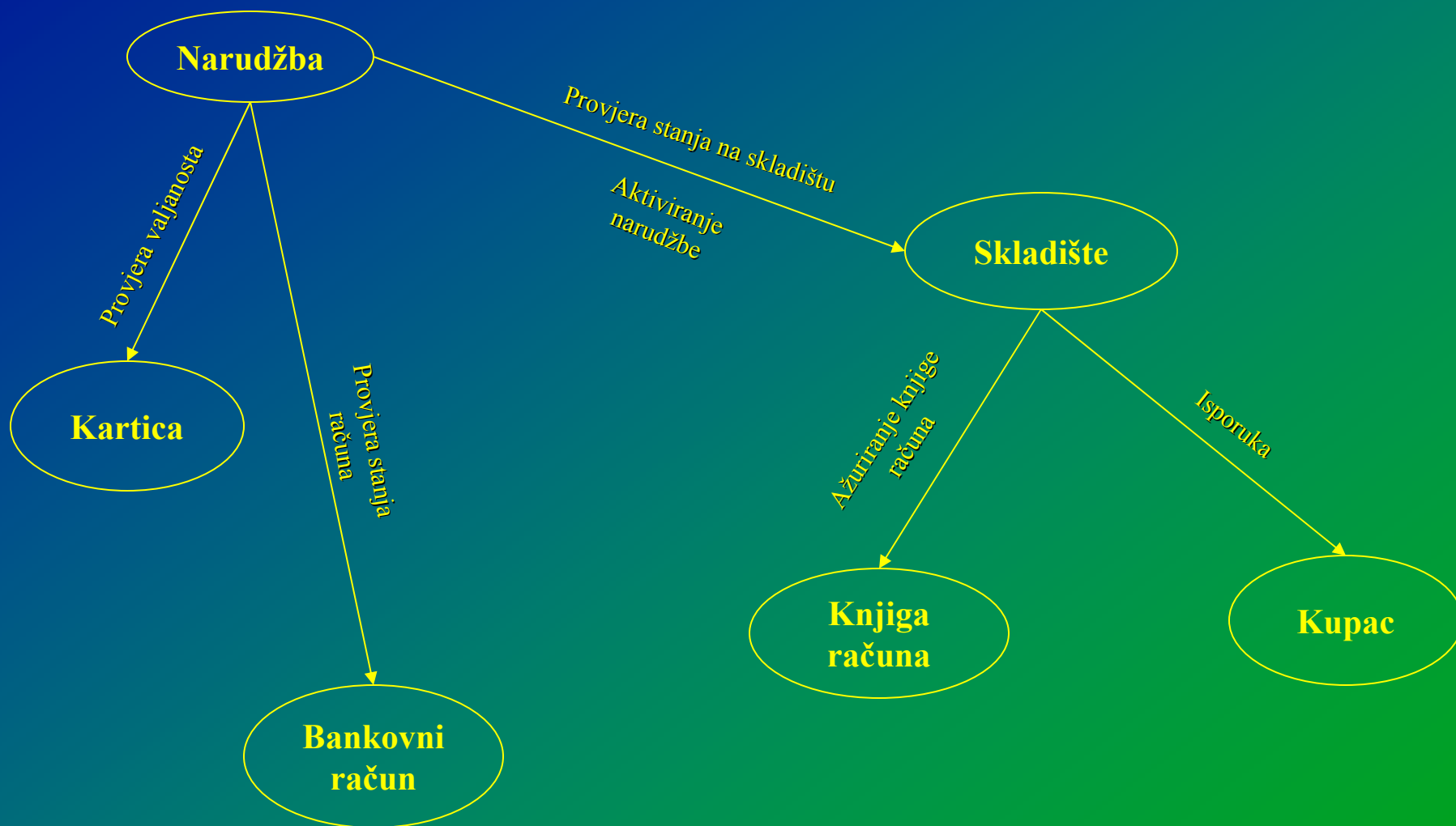
● objektno orijentirano oblikovanje (*object-oriented design*)

- sistem razlaže na objekte koji međusobno komuniciraju

Algoritamska dekompozicija



Objektna dekompozicija



Objektni model

■ Vrste programskih paradigmi

- proceduralno orijentirana
- logički orijentirana
- orijentirana funkcijama
- orijentirana pravilima
- orijentirana ograničenjima
- objektno orijentirana

algoritmi

ciljevi, predikatni račun

definicije i pozivi funkcija

ako-tada pravila

odnosi među varijablama

klase i objekti

- Objektno orijentirano programiranje - OOP
 - OOP je način ugradnje programske podrške kod kojeg su programi organizirani kao kooperativne skupine objekata, od kojih svaki predstavlja pojavnost neke klase, a sve klase su članovi neke hijerarhije klasa sjedinjene preko nasljeđivanja
 - Programiranje bez nasljeđivanja nije objektno orijentirano, već je riječ o programiranju s apstraktnim tipovima podataka
 - Osnovni gradivni element jezika je objekt a ne algoritam
- Jezik je objektno orijentiran ako i samo ako:
 - podržava objekte koji predstavljaju apstrakciju stvarnog svijeta sa skrivenim lokalnim stanjem i sa interfejsom koji definira operacije nad tim objektima
 - objekti pripadaju nekoj klasi (*class*)
 - klase mogu nasljeđivati svojstva od nadklasa
- Za jezik bez nasljeđivanja kaže se da je zasnovan na objektima (*object-based*)

- Objektno orijentirano oblikovanje (*object-oriented design, OOD*)
 - Metoda oblikovanja koja sadržava proces objektno orijetirane dekompozicije i notaciju koja prikazuje kako logičke i fizičke, tako i statičke i dinamičke modele

- Objektno orijentirana analiza (*object-oriented analysis, OOA*)
 - Metoda analize koja ispituje zahtjeve iz perspektive klasa i objekata služeći se rječnikom iz domene problema

Elementi objektnog modela

- Osnovni elementi objektnog modela
 - moraju postojati da bi model bio objektno orijentiran
 - apstrakcija (*abstraction*)
 - ućahurivanje (*encapsulation*)
 - modularnost (*modularity*)
 - hijerarhija (*hierarchy*)
- Sporedni elementi objektnog modela
 - tipizacija (*typing*)
 - istodobnost (*concurrency*)
 - postojanost (*persistence*)

■ Apstrakcija podataka

- zanemarivanje nebitnih dijelova objekta i koncentriranje na one koji ga razlikuju od ostalih objekata
- potrebna radi lakšeg poimanja kompleksnih sistema

■ Vrste apstrakcije

redom od najkorisnije prema manje korisnima:

- apstrakcija entiteta - objekt koji predstavlja neki model iz područja problema
- apstrakcija akcije - objekt koji sadržava skup operacija
- apstrakcija virtualnog stroja - objekt koji grupira operacije koje se koriste na nekom višem stupnju kontrole
- slučajna apstrakcija - objekt koji grupira skup operacija među kojima ne postoji povezanost

Nastojat će se graditi apstrakcija entiteta, jer se time koristi rječnik iz domene problema.

■ Terminologija

- Klijent - objekt koji koristi resurse nekog drugog objekta, poslužitelja (*server*)
- Ponašanje objekta definirano je uslugama koje objekt može obaviti za druge objekte i radnjama koje taj objekt može obaviti nad drugim objektima.
- Ugovorni model programiranja - definiraju se odgovornosti objekta (Meyer B., Object oriented software construction, Prentice Hall, New York, 1988.)
- Protokol - skup i redoslijed postupaka koje klijent može obaviti nad objektom
- Postupci - *operations* (Ada), *methods* (Smalltalk), *member functions* (C++)

■ Primjer apstrakcije: Ekran za kupca

C++  EkranZaKupca

```
class cEkranZaKupca {
public:
    cEkranZaKupca(int Port);
    ~cEkranZaKupca();

    bool    OK();
    bool    Obrisi();
    bool    SetGornjiRed(char Red[]);
    bool    SetDonjiRed(char Red[]);

private:
    bool    Parametri(DWORD BaudRate, BYTE ByteSize,
                      BYTE Parity, BYTE StopBits);
    bool    Ispis(char Niz[]);
    bool    Ispis(char Niz[], unsigned int BrojZnakova);

    HANDLE  PortHandle;
    char    GornjiRed[20+1];
    char    DonjiRed[20+1];
    DWORD   PosljednjaPogreska;
    DWORD   BaudRate;
    BYTE    ByteSize, Parity, StopBits;
};
```


■ Primjer apstrakcije: kreditna kartica

C++  KreditnaKarticaLosa

```
class      cKreditnaKartica {
public:
    cKreditnaKartica(char ID[]);
    cKreditnaKartica(char ID[], char Datum[]);

    bool      operator==(cKreditnaKartica &Kartica);
    char *    GetIDKartice();
    char *    GetDatumIsteka();

private:
    char      IDKartice[MAXID+1];
    char      DatumIsteka[MAXDATUM+1];
};
```

Nije dobro, jer se može napraviti

```
strcpy (Kartical.GetIDKartice(), "2");
```

■ Učahurivanje (*encapsulation*)

- niti jedan dio složenog sistema ne bi smio ovisiti o unutrašnjim detaljima drugoga dijela
- robustnost se postiže skrivanjem podataka i detalja ugradnje
- komplementaran koncept u odnosu na apstrakciju
 - apstrakcija se fokusira na vidljivo (*observable*) ponašanje objekta, dok se enkapsulacija fokusira na ugradnju koja podržava takvo ponašanje
 - enkapsulacija omogućava eksplicitnu granicu između različitih apstrakcija
- opis klasa ima dva dijela : interfejs (*interface*) i ugradnju (*implementation*).
 - interfejs predstavlja vanjski pogled na klasu
 - ugradnja klasa predstavlja mehanizme pomoću kojih su ostvareni postupci opisani u interfejsu

■ Primjer: poboljšana kreditna kartica

C++  KreditnaKartica

```
class      cKreditnaKartica {
public:
    cKreditnaKartica(string ID);
    cKreditnaKartica(string ID, string Datum);

    bool      operator==(cKreditnaKartica &Kartica);
    string     GetIDKartice();
    string     GetDatumIsteka();
private:
    string     IDKartice;
    string     DatumIsteka;
};
```

- Primjer: C++  NarucivanjeKnjiga



■ Modularnost

- razlaganje sistema u skup relativno nezavisnih modula koji objedinjuju klase koji imaju nešto zajedničko
- naš primjer : identifikacija skupa klasa i objedinjavanje njihovih deklaracija i definicija u zajedničke datoteke
 - `Skladiste.h` i `Skladiste.cpp` - interfejs i ugradnja za klase :
 - `Skladiste`
 - `Knjiga`
 - `KnjigaRacuna`
 - `Racun`
 - slično bi se klase
 - `KreditnaKartica`
 - `Banka`mogle objediniti u `Placanje.h` i `Placanje.cpp`

■ Hijerarhija

- skup apstrakcija često formira hijerarhiju
- definiramo hijerarhiju:

Hijerarhija predstavlja rangiranje apstrakcija

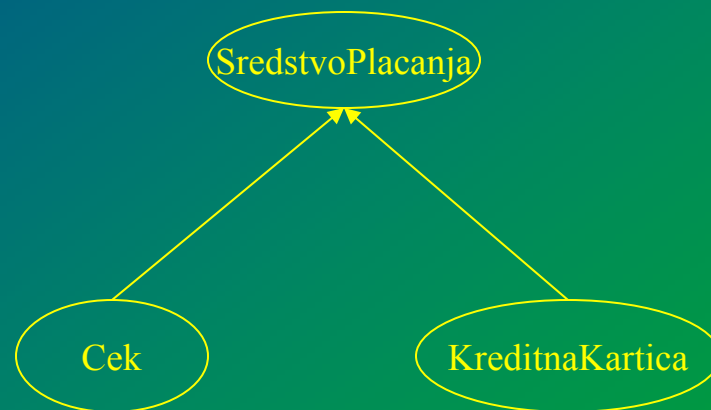
- dvije najvažnije hijerarhije su:
 - struktura klasa ("je (is a)" hijerarhija)
 - struktura objekata ("dio od (part of)" hijerarhija)
- primjer hijerarhije : jednostruko nasljeđivanje
 - nasljeđivanje definira odnose među klasama, gdje jedna klasa dijeli strukturu i ponašanje jednog ili više drugih klasa (jednostruko i višestruko nasljeđivanje)
 - "je tip (is a kind of)" veza između nadređene i podređene klase
 - predstavlja hijerarhiju generalizacija/specijalizacija

Jednostruko nasljeđivanje

```
class cSredstvoPlacanja {  
public:  
    ...    // implementacija  
protected :  
    double      Iznos;  
    string      Datum;  
}
```

```
class cCek : public cSredstvoPlacanja {  
public:  
    ...    // implementacija  
private:  
    string      BrojCeka;  
    string      UplacenUKorist;  
}
```

```
class cKreditnaKartica : public cSredstvoPlacanja {  
public:  
    ...    // implementacija  
private:  
    string      ID;  
    string      DatumIsteka;  
}
```



- tijekom razvoja hijerarhija nasljeđivanja, struktura i ponašanje koji su zajednički za više klasa, prelaze u zajedničku nadklasu (*superclass*)
- principi apstrakcije, enkapsulacije i hijerarhije međusobno su u konfliktu
 - enkapsulacija podataka postavlja barijeru iza koje su sakrivene metode i stanja objekta, dok nasljeđivanje zahtijeva otvaranje interfejsa i mogućnost pristupa stanju i metodama bez enkapsulacije
 - dvije vrste klijenta klase :
 - objekti koji pozivaju operacije nad instancama klase
 - podklase koje nasljeđuju klase
- u C++ deklaratori:
 - *private* - članovi dostupni samo samoj klasi
 - *protected* - članovi dostupni klasi i njenim podklasama
 - *public* - članovi dostupni svima

■ Višestruko nasljeđivanje

- u nekim situacijama je dobro imati mogućnost nasljeđivanja od više različitih klasa
- Primjer : sistem za simulaciju prometa u gradu s ciljem dobivanja vjerojatnih vremena dolaska hitnih vozila (*emergency*) na odredište

```
class cVozilo {  
protected:  
    double          Brzina;  
    struct Position  Polozaj;  
    struct Position  CiljPuti;  
};
```

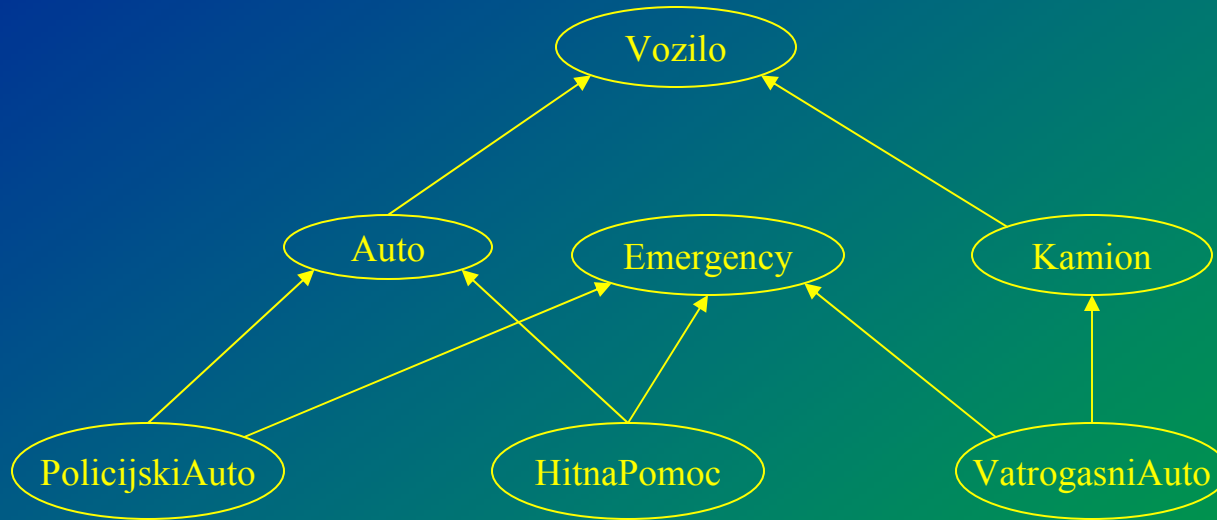
```
class cEmergency {  
protected:  
    bool    UpaljenaSirena;  
};
```

```
class cPolicijskiAuto {  
};
```

```
class cHitnaPomoc {  
};
```

```
class cVatrogasniAuto {  
};
```

Višestruko nasljeđivanje



```
class cVozilo { };  
class cEmergency { };  
class cAuto : public cVozilo{ };  
class cKamion : public cVozilo{ };  
class cPolicijskiAuto : public cAuto, public cEmergency { };  
class cHitnaPomoc : public cAuto, public cEmergency { };  
class cVatrogasniAuto : public cKamion, public cEmergency { };
```

- često postoji mogućnost izbora između korištenja nasljeđivanja i korištenja članskih varijabli da bi se predstavio neki odnos

```
class cEmergency { };
```

```
class cVehicle {  
public:  
    cEmergency *eptr;  
}
```

```
class cAuto : public cVozilo{ };  
class cKamion : public cVozilo{ };  
class cPolicijskiAuto : public cAuto { };  
class cHitnaPomoc : public cAuto { };  
class cVatrogasniAuto : public cKamion { };
```

```
cAuto::cAuto {  
    eptr = NULL;  
}  
cPolicijskiAuto::cPolicijskiAuto {  
    eptr = new Emergency;  
}
```

- jednostavna konverzija iz emergency vozila u obično vozilo (eptr = NULL)

■ Sadržavanje (*aggregation*)

- sadržavanje se prepoznaje po “dio od” odnosu među klasama
- nije karakteristična samo za OOP (strukture u ostalim programskim jezicima)

```
class cMotor {  
    ...  
public:  
    double    Zapremina;  
    int       BrojCilindara;  
}  
  
class cVozilo {  
    ...  
private:  
    cMotor    Motor;  
}
```

- pitanje vlasništva nad agregatnim objektima
 - sadržavanje po vrijednosti
 - sadržavanje po referenci

■ Tipizacija

- “Tip je precizna karakterizacija strukture ili ponašanja koju dijeli neki skup entiteta”
- “Tipizacija je poštivanje klasa objekta, tako da se objekti različitih tipova ne mogu međusobno zamjenjivati, ili se mogu zamjenjivati na vrlo ograničen način”
- stroga i slaba tipizacija (*strong* & *weak typing*)
 - **stroga tipizacija**: usporedba tipova se strogo provodi (narušavanje uskladiivosti tipova otkriva se prilikom prevođenja)
 - **slaba tipizacija**: klijent može slati bilo kakve poruke drugim klasama, čak iako klasa ne zna kako ih interpretirati (pogreške se otkrivaju prilikom izvođenja programa)
 - uz strogu tipizaciju oblikovanje sistema ovisi o tipovima podataka
 - loša strana je što se unose semantičke ovisnosti tako da sasvim male promjene u baznoj klasi mogu zahtijevati promjenu i ponovno prevođenje svih podklasa
 - u slučaju nepostojanja parametriziranih klasa (*templates*), teško se mogu implementirati generički (*type-safe*) skupovi objekata (potrebno je koristiti `void` pokazivače)

```

class cVozilo{
public:
    double    TrenutnaBrzina ();
    void      UpaliMotor ();

    virtual void    UbaciUVisuBrzinu ();
    virtual void    UbaciUNizuBrzinu ();
protected:
    double    Brzina;
    string    ImeTipa;
}

class cKamion : public cVozilo {
public:
    void      UkrcajTeret (double Tezina);
    double    TezinaUkrcanogTereta();

    void      UbaciUVisuBrzinu ();
    void      UbaciUNizuBrzinu ();
private:
    double    TezinaTereta, MaxDopustenaTezina;
}

class cBus : public cVozilo {
public:
    void      UkrcajPutnike (int Broj);

    void      UbaciUVisuBrzinu ();
    void      UbaciUNizuBrzinu ();
private :
    int       BrojPutnika, MaxBrojPutnika;
}

```

```

cVozilo  v1, v2;
cKamion  kam;
cBus     bus;

double  brz = v1.TrenutnaBrzina();
kam.UkrcajTeret(1000);
bus.UkrcajPutnike(25);

v1.UkrcajTeret(100);    // nelegalno, bazna klasa nema UkrcajTeret()
bus.UkrcajTeret(100);  // nelegalno, Bus nema UkrcajTeret()

bus.UpaliMotor();

```

pridruživanje varijabli (primjer stroge tipizacije)

```

v1 = v2;
v1 = kam;    // odbacuju se svojstva koje nema cVozilo

bus = v1;    // nelegalno
bus = kam;   // nelegalno

```

možemo obaviti konverziju jednog tipa u drugi, ako smo sigurni da je argument nad kojim obavljamo konverziju stvarno danog tipa

```

void NekaFunkcija (cVozilo &v)
...
if( ((cKamion&)v).TezinaUkrcanogTereta() > 20000 )
...

```

- statičko i dinamičko povezivanje (*static (early)* & *dynamic (late)* *binding*) - odnosi se na trenutak kada se imena povezuju s tipovima
 - statičko povezivanje : tipovi svih varijabli i izraza su poznati prilikom prevođenja
 - dinamičko povezivanje : poznati tek kod izvođenja
- različit koncept u odnosu na strogu i slabu tipizaciju (jezik može biti slabo tipiziran sa statičkim povezivanjem, i obrnuto)

```
int Prestigni (CVozilo &v1, CVozilo &v2)
    if (v1.TrenutnaBrzina() > v2.TrenutnaBrzina())
        v2.UbaciUVisuBrzinu();
```

- ne zna se unaprijed (prilikom prevođenja) koje klase će biti v1 i v2
- ali, bez obzira kojeg tipa bili v1 i v2 poziv funkcije TrenutnaBrzina() će biti statički povezan (biti će pozvana funkcija iz bazne klase)
- za razliku od toga pozivanje funkcije UbaciUVisuBrzinu() će ovisiti o tome kojeg su tipa varijable v1 i v2, te će biti dinamički povezan (odrediti će se prilikom izvođenja programa)
- primjer višeobličja (*polymorphism*)

- Istodobnost (*concurrency*)
 - pojam procesa (podproces, nit)
 - prava istodobnost - višeprosorsko računalo
 - simulirana istodobnost - jednoprocesorsko računalo
 - aktivni i neaktivni objekti
 - podržana od jezika:
 - Smalltalk - klasa `Process` koji se dalje nasljeđuje
 - Ada - posao (*task*)
 - VB - *threading model, thread per object / thread pool*
 - u C++ se implementira preko dodatnih biblioteka (*library*) koje implementiraju klase `Sched`, `Timer`, `Task` i druge (AT&T), u Javi klasa `Thread`.
 - MS Windows `AfxCreateThread(int (*f) (...))`

■ Postojanost (*persistence*)

- svojstvo objekta preko kojeg se njegovo postojanje transcendirira u vremenu i prostoru
- različite vrste postojanosti objekata :
 - prijelazni rezultati u izračunavanjima izraza
 - lokalne varijable kod aktiviranja procedura
 - globalne varijable
 - podaci koji postoje između izvođenja programa
 - podaci koji postoje između različitih verzija programa
 - podaci koji nadžive program
- Primjer: postojani objekt postoji između izvođenja programa

C++  Postojanost

C++  PostojanostFileStream

Prednosti objektnog modela

- Što će nam objektno orijentirano programiranje kada sve to možemo napisati i u običnom C-u ???
 - apstrakcija podataka (*data abstraction*)
 - ponovna iskoristivost koda (*code reusability*)
 - jednom napisani kod se koristi više puta i na više različitih načina (npr. lista se može realizirati tako da se definira javno interfejs a da se tipovi objekata koji će se u njoj pohraniti definiraju naknadno = **parametrizirani tipovi**)
 - ukoliko je potrebno promijeniti ili dodati neke postupke u već definiranu klasu, to se izvodi jednostavno pomoću **nasljeđivanja** i **višeobličja**
 - sistemi se grade na osnovi stabilnih međuforni, tako da sistem evoluira tokom vremena

klase i objekti

- **Objekt:** pojedinačni stvarni ili apstraktni entitet, s dobro definiranom ulogom u području problema
 - općenitije - objekt je sve što ima dobro definiranu granicu
 - atributi (vrijeme, težina, boja) ne predstavljaju objekte, ali mogu biti svojstva drugih objekata
 - Objekt ima stanje, ponašanje i identitet
 - Struktura i ponašanje sličnih objekata definirani su u njihovom zajedničkom klasi
 - Pojavnost (*instance*) i objekt su sinonimi

■ Stanje

- stanje objekta sadrži sva (obično statička) svojstva objekta i trenutne (obično dinamičke) vrijednosti svakog od ovih svojstava
- sva svojstva imaju neku vrijednost
 - može biti jednostavna varijabla, ili može označavati drugi objekt
- objekti postoje u vremenu, promjenjivi su, imaju stanje, stvaraju se i uništavaju, te mogu biti dijeljeni
- budući da ima stanje, svaki objekt zauzima određeni prostor

```
struct Osoba {  
    char JMBG[13+1];  
    char Prezime[40+1];  
    char Ime[40+1];  
    float Placa;  
}
```

- dobro je učahuriti svojstva objekta

```
class cOsoba {  
public:  
    string GetJMBG();  
    string GetPrezime();  
    string GetIme();  
    float  GetPlaca();  
protected:  
    string JMBG;  
    string Prezime;  
    string Ime;  
    float  Placa;  
}
```

- Gornjom deklaracijom nije načinjena pojavnost objekta.

```
cOsoba Pero, Ante, Ivica, Marica;
```

■ Ponašanje

- kako objekt djeluje i reagira s obzirom na promjene stanja i protok poruka
- u čistim objektno-orijentiranim jezicima (Smalltalk) govori se o objektu koji predaje poruku drugom objektu, a u C++ se govori o objektu koji poziva člansku funkciju drugog objekta
- operacije koje klijenti mogu obaviti na objektu zovemo postupcima. Postupci su dio deklaracije klasa
- stanje objekta predstavlja kumulativni rezultat njegovog ponašanja

- postupci (*methods, member functions*) - usluge koje klasa nudi klijentima
 - ***konstruktor*** - postupak koja stvara objekt i inicijalizira njegovo stanje
 - ***destruktor*** - postupak koja uništava objekt oslobađajući zauzeti prostor
 - ***modifikator*** - postupak koja mijenja stanje objekta
 - ***selektor*** - postupak koja pristupa stanju objekta, ali ga ne mijenja
 - ***iterator*** - postupak koja dozvoljava da se pristupi svim dijelovima objekta u nekom definiranom redoslijedu
- postupci se u nekim jezicima mogu ostvariti kao slobodni potprogrami (dakle ne kao postupci neke klase)
 - korisno kada postupak djeluje nad različitim objektima ili različitim klasama (teško se odlučiti kojem bi objektu ili kojoj klasi pripadao)

■ Identitet

- svojstvo objekta koje ga razlikuje od svih drugih objekata
- potrebno je razlikovati ime objekta od samog objekta

```
struct Point {  
    int x, y;  
    Point() : x(0), y(0) {}  
    Point(int xValue, int yValue) : x(xValue), y(yValue) {}  
};
```

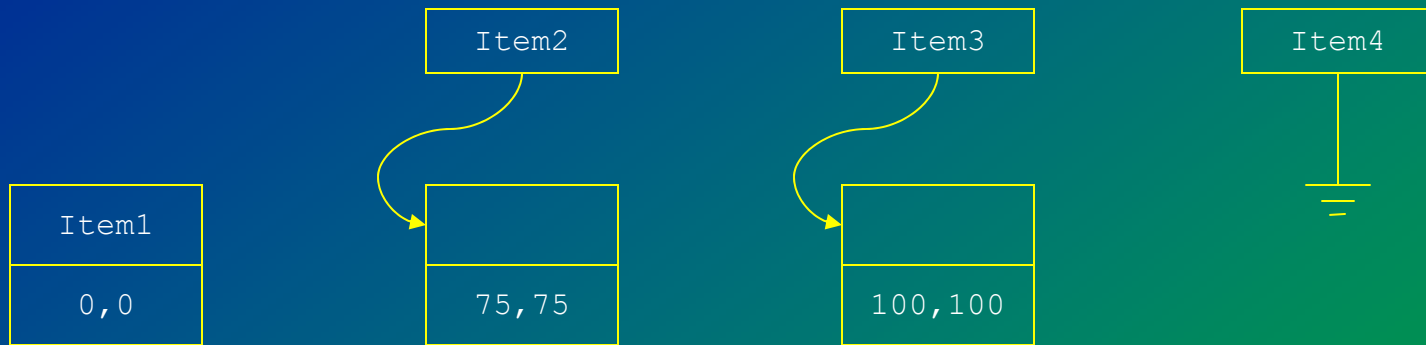
- Tačka je deklarirana kao zapis, a ne klasa. Uobičajeno kada se radi o apstrakciji koja nema vlastito ponašanje, već je samo skup podataka

```
class CDisplayItem {  
public:  
    CDisplayItem();  
    CDisplayItem(const Point & Location);  
  
    virtual void Draw();  
    virtual void Move(const Point & Location);  
  
    Point Location() const;  
protected:  
    ...  
};
```

```

cDisplayItem Item1;
cDisplayItem * Item2 = new cDisplayItem(Point(75,75));
cDisplayItem * Item3 = new cDisplayItem(Point(100,100));
cDisplayItem * Item4 = 0;

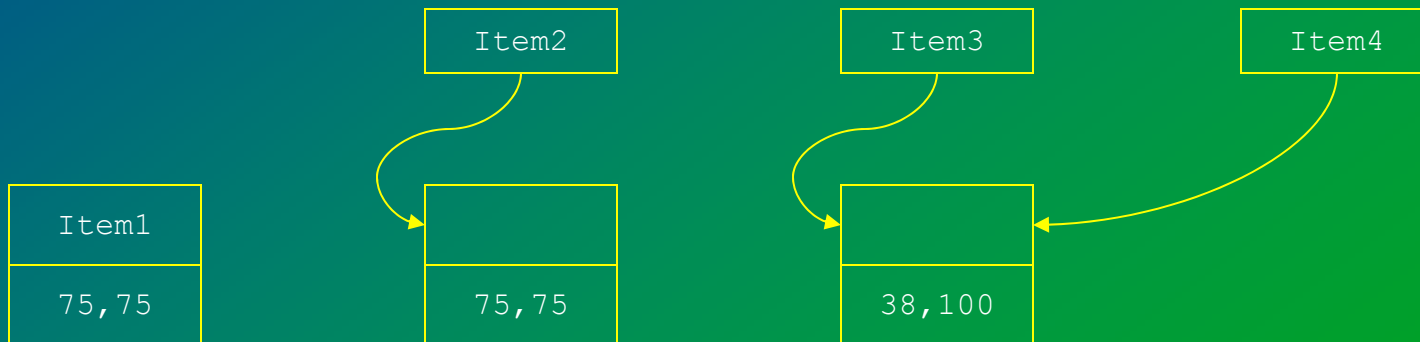
```



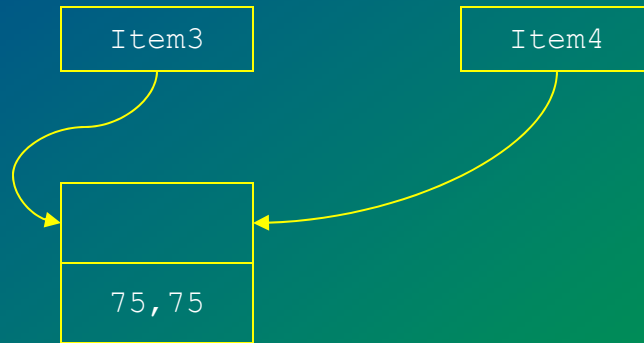
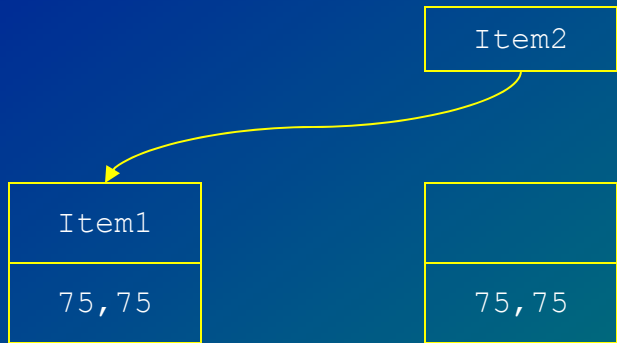
```

Item1.Move(Item2->location);
Item4 = Item3;
Item4->Move(Point(38,100));

```



```
Item2 = &Item1; // alias  
Item4->Move (Item2->Location);
```



Nastalo je smeće (*garbage*).

■ Kopiranje, pridjeljivanje, jednakost

● Kopiranje

```
void Highlight(DisplayItem& i);
```

- nakon poziva npr. `Highlight (Item1)` u funkciji stvara se drugo ime (*alias*) za `Item1`, formalni parametar `i` označava referencu na objekt označen aktualnim parametrom

```
void Drag(DisplayItem i);
```

- stvara se kopija stvarnog parametra, pa nema aliasa; `i` označava drugi objekt (ali sa istim stanjem)
- preporučljivo je, zbog performansi, predavati reference kao parametre. Stvarni argument može se zaštititi od promjene ključnom riječju `const`
- ukoliko je potrebno imati kontrolu nad predavanjem kopija, u C++ se može definirati operator kopiranja (tzv. *copy constructor*):

```
DisplayItem(const DisplayItem&);
```
- može biti pozvan eksplicitno (kada se deklarira objekt), ili implicitno (prilikom poziva funkcije po vrijednosti)

● Pridjeljivanje

- semantika pridjeljivanja se također može eksplicitno definirati

```
virtual DisplayItem& operator=(const DisplayItem&);
```

- bez definiranja operatora, poziva se pretpostavljeni (*default*) operator koji obavlja kopiranje po članovima

● Jednakost

- dvije mogućnosti :
 - jednakost može značiti da dva različita imena označavaju isti objekt
 - jednakost može značiti da dva različita objekta imaju isto stanje
- u C++ je moguće definirati semantiku operatora jednakosti prekrivanjem operatora == ili !=

```
virtual int operator==(const DisplayItem&) const;  
int operator!=(const DisplayItem&) const;
```

- virtual zato što se od podklasa očekuje da redefiniraju značenje
- nejednakost nije virtual zato što se želi da ona uvijek bude logička negacija jednakosti

● Primjer: C++ Matrice

Odnosi među objektima

■ Veza (*link*)

- fizička ili konceptualna povezanost među objektima
- veza označava određenu asocijaciju preko koje jedan objekt (klijent) primjenjuje uslugu (*service*) drugog objekta (poslužitelja)
- Kao sudionik u vezi objekt može biti
 - **actor** - objekt koji može djelovati na druge objekte, ali na kojeg se nikad ne djeluje
 - **server** - objekt koji nikad ne djeluje na druge objekte; jedino drugi objekti djeluju na njega
 - **agent** - objekt koji može djelovati i na kojeg se može djelovati; obično se stvara da bi obavio neki posao za actora ili drugog agenta

- Vidljivost - ukoliko postoji veza između objekata A i B i poruke se šalju od A do B, objekt B mora biti vidljiv objektu A
 - B je globalan objektu A
 - B je parametar nekom postupku klijenta
 - B je dio objekta klijenta
 - B je lokalno deklariran objekt u nekom postupku klijenta

- Sinkronizacija
 - kad jedan objekt šalje poruku drugom preko veze, kaže se da su objekti sinhronizirani
 - problem kod višenitnog (višeprocenog, *multithreading*) izvršavanja

■ Sadržavanje

- označava odnos cjelina/dio, s mogućnošću pomicanja sa cjeline (agregat) na njene dijelove (atributi)
- primjer vozila kojeg je motor sastavni dio (sadržavanje po vrijednosti)

```
class cMotor {  
public:  
    ...  
};  
  
class cVozilo {  
public:  
    void UpaliMotor();  
    ...  
private:  
    cMotor    Motor;  
};
```

- moguće je i sadržavanje po *referenci* (objekt ne sadrži drugi objekt, već pokazivač na drugi objekt)

Priroda klasa

■ klasa

- skup objekata koji dijele zajedničku strukturu i zajedničko ponašanje
- razlika između vanjskog i unutrašnjeg pogleda na klasu
 - **interfejs** (*interface*) - osigurava vanjski izgled i naglašava apstrakciju istovremeno sakrivajući strukturu i njeno ponašanje; sastoji se prvenstveno od deklaracija postupaka primjenjivih na pojavnosti te klase.
 - **ugradnja** (*implementation*)- unutrašnji izgled klase; sastoji se od ugradnje svih postupaka interfejsi
- interfejs možemo podijeliti :
 - *public* : deklaracija dostupna svim klijentima
 - *protected* : deklaracija dostupna samo klasi, podklasama i prijateljima (*friends*)
 - *private* : deklaracija dostupna samo klasi i prijateljima (*friends*)

Odnosi među klasama

- Tri osnovne vrste odnosa među klasama
 - generalizacija/specijalizacija - označava odnos "jest" (*is a kind of*)
 - cjelina/dio - označava odnos "jest dio od" (*is a part of*)
 - asocijacija - označava semantičku zavisnost među inače nepovezanim klasama
- Podržane vrste u OO-jezicima:
 - asocijacija
 - nasljeđivanje
 - sadržavanje
 - korištenje
 - parametrizacija
 - metaclass

Asocijacija

- Označava semantičku zavisnost - ali ne govori ništa o smjeru zavisnosti niti o međusobnom odnosu klasa

- **Primjer: prodaja**

```
class cProduct; class cSale;  
class cProduct {  
    ...  
protected:  
    cSale *LastSale;  
};  
  
class cSale {  
    ...  
protected:  
    cProduct **ProductSold;  
};
```



- **Kardinalnost asocijacije**

- 1:1
- 1:n
- n:n

Nasljeđivanje

■ Primjer: sistem za prenošenje podataka sa sonde

```
class cTelemetrijskiPodaci {
public:
    cTelemetrijskiPodaci();
    virtual ~cTelemetrijskiPodaci();

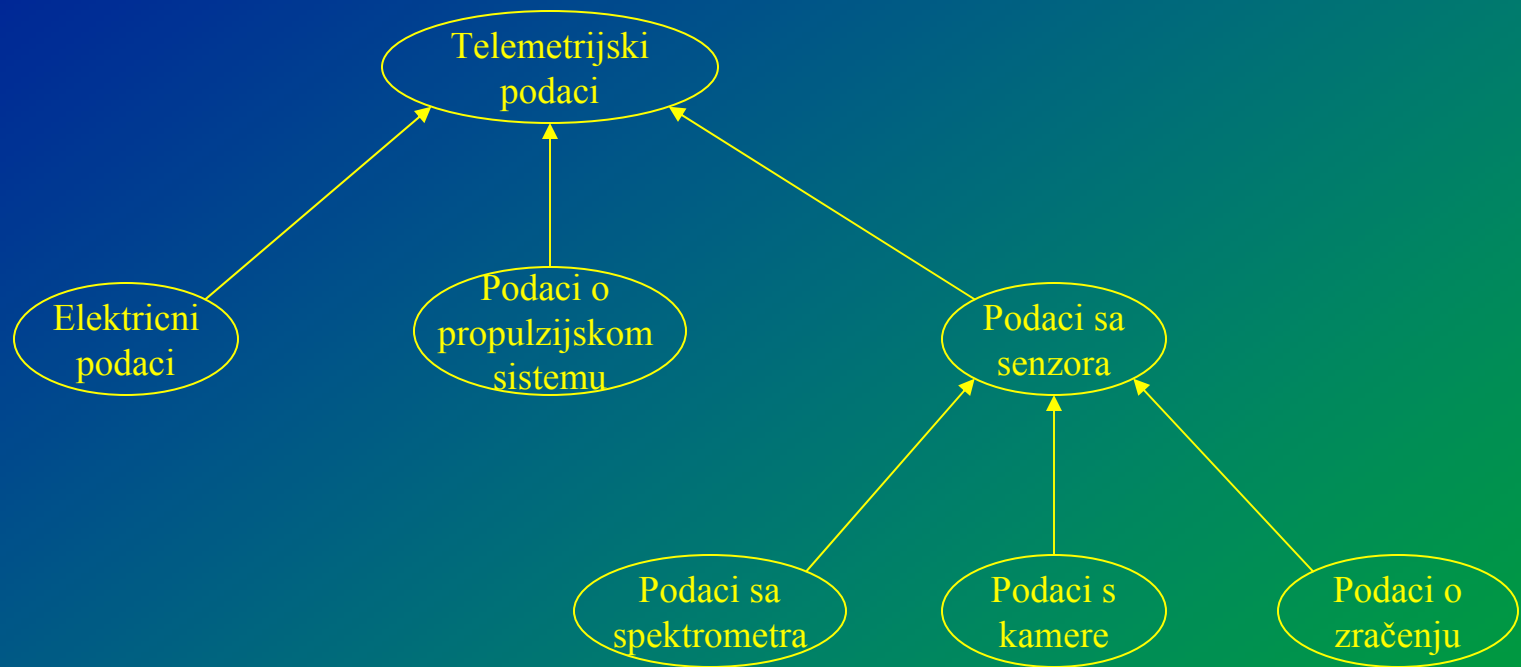
    virtual void Posalji();

    Time      TrenutnoVrijeme() const;
protected:
    int       Id;
    Time      OznakaVremena;
};

class cElektricniPodaci : public cTelemetrijskiPodaci {
public:
    cElektricniPodaci(float U1, float U2, float I1, float I2);
    ~cElektricniPodaci();

    void Posalji();

    float      TrenutnaSnaga() const;
protected:
    float      NaponCeliije1, NaponCeliije2;
    float      StrujaCeliije1, StrujaCeliije2;
};
```



- neki klase neće imati instance (*apstraktne klase*), dok se iz nekih neće izvoditi podklase (*konkretne klase*)
- apstraktne klase očekuju da će podklase dodati strukturu i ponašanje, dopunjavajući ugradnju postupaka apstraktne klase

- u C++ se može deklarirati *čista virtualna funkcija*. Iz klase koja sadrži takvu funkciju ne može se instancirati objekt.

```
class cShape {  
public:  
    virtual void Draw() = 0;  
};
```

```
class cRectangle: public cShape {  
public:  
    void Draw();  
};
```

- Osnovne klase - klase na vrhu hijerarhijske strukture.
 - C++: više osnovnih klasa
 - Smalltalk: osnovne klase `Object`

Jednostruko višebličje (polimorfizam)

```
void cTelemetrijskiPodaci::Posalji() {  
    // pošalji Id  
    // pošalji OznakuVremena  
}  
  
void cElektricniPodaci::Posalji() {  
    TelemetrijskiPodaci::Posalji();  
    // pošalji napone  
    // pošalji struje  
}  
  
cTelemetrijskiPodaci    Tel;  
cElektricniPodaci       El(5.0, -5.0, 3.0, 7.0);  
  
void PosaljiSvjezePodatke(cTelemetrijskiPodaci &d, const Time &t){  
    if (d.TrenutnoVrijeme() >= t)  
        d.Posalji();  
}  
  
PosaljiSvjezePodatke(Tel, Time(60));  
PosaljiSvjezePodatke(El, Time(120));
```

Nasljeđivanje i tipizacija

```
Tel = El;  
El  = Tel;           // ne može
```

Višestruko višeeobličje

- u klasi `cDisplayItem` imali smo funkciju

```
virtual void Draw();
```

- za različite objekte koji se mogu prikazati (izvedene iz klase `cDisplayItem`), pozivati će se različite funkcije
- problem se javlja ukoliko želimo pozivati različite funkcije ovisno i o tome kakav je uređaj na koji se iscrtava (npr. tekst, grafika..)
- potrebno je višeeobličje u više nego jednom parametru
- u C++ se rješava tehnikom *double dispatching*
- definiramo još jednu osnovnu klasu `cDisplayDevice` i njene podklase

```
void Rectangle::Draw(cDisplayDevice& DD) {  
    DD.Line (X0,Y0, X1,Y1);  
    ...  
}
```

- najprije se očituje višeeobličje prilikom poziva funkcije `Draw`, na osnovu točnog tipa objekta `cDisplayItem` za kojeg je funkcija `Draw` pozvana, a zatim se očituje višeeoblično ponašanje na osnovu točnog tipa podklasa od `cDisplayDevice`

Sadržavanje

■ Po vrijednosti (*by value*)

```
class cMotor{  
    ...  
};  
  
class cVozilo {  
    ...  
private:  
    cMotor    Engine;  
}
```



■ Po referenci (*by reference*)

```
class cKreditnaKartica {  
    ... ugradnja i interfejs  
}  
  
class cBanka {  
    ... interfejs  
private:  
    cKreditnaKartica **Kartice;  
}
```

Korištenje

- klasa klijenta ovisi o klasi koja osigurava neku uslugu (*service*)

```
class cBanka{  
public:  
    cBanka();  
    ~cBanka();  
  
    bool    ProvjeriValjanost(cKreditnaKartica &kart);  
  
private:  
    int      BrojKartica;  
    cKreditnaKartica **Kartice;    // sadržavanje po referenci  
};
```



Parametrizacija

- Kako ugraditi stog da radi sa proizvoljnim tipovima?

- Generičko programiranje

```
template<class _Type> class cStog {  
public:  
    cStog();  
    cStog(int BrojElem);  
    bool    Push(_Type);  
    _Type    Pop();  
private:  
    _Type    *Rep;  
    int      MaxBrojElem;  
    int      BrojElem;  
};
```

```
cStog<int>    CjelobrojniStog;  
cStog<float> RealniStog;
```

Metarazred

- Objekt je pojavnost neke klase
- Metaklasa omogućava da klasa kao takva bude objekt
- Instance metaklase su klase

Vrste klasa

- Raznovrsnost uloga koje klase mogu imati vodi do različitih vrsta klasa, koje su specijalizirane za neku posebnu svrhu
 - konkretni tipovi
 - apstraktni tipovi
 - čvorovi
 - akcije
 - interfejsi
 - oznake (*handles*)
 - kosturi programa (*application frameworks*)
- različite vrste klasa nemaju različitu sintaksu u programskom jeziku, već odražavaju odluke o dizajnu

Konkretni tipovi

- klase su konkretne u smislu da svaka predstavlja relativno jednostavan koncept sa svim operacijama koje podržavaju taj koncept (npr. klase `cVector`, `cList`, `cDatum`, `cComplex`)
- odnos između ugradnje i interfejsi je 1-prema-1
- nisu predviđeni za daljnje nasljeđivanje
- klasa koja predstavlja konkretni tip mora:
 - biti blizak danom konceptu
 - osigurati efikasno izvođenje i zauzimanje prostora
 - imati minimalnu ovisnost o drugim klasama
 - biti shvatljiv i koristan u izolaciji
- da bi iskoristio konkretni tip, korisnik mora razumjeti njegove pripadne detalje (ne postoje opća svojstva koja vrijede za sve konkretne tipove)
- dobro je i dalje skrivati ugrađene detalje
- zbog maksimalne efikasnosti dobro je koristiti `inline` funkcije)
 - **Primjeri:** C++ `Lista`, `Vector`

■ Ponovno iskorištavanje konkretnih tipova

- konkretni tipovi gotovo nikad nisu osnova za daljnje nasljeđivanje
- ali, mogu se koristiti kao i ostali ugrađeni tipovi

```
class csistemJednadzbi {  
private:  
    cMatrica A;  
    cVector  b;  
    cVector  x;  
};
```

ukoliko se želi imati mogućnost izvođenja tipova:

```
class cMatrica {  
public:  
    // interfejs, sadrži virtualne funkcije, npr.  
    virtual int GetReda () { return Reda; }  
    virtual int GetStupaca () { return Stupaca; }  
protected:  
    // ostali detalji ugradnje, možda i poneko svojstvo  
    int Reda, Stupaca;  
private:  
    // svojstva  
    float **Mat;  
};
```

● Izvedene klase

```
class cKvadratnaMatrica: public cMatrica {
public:
    // interfejs
    cKvadratnaMatrica (int inRang) : Rang(inRang),
                                     cMatrica (Rang, Rang) {}

    int GetReda () { return Rang; }
    int GetStupaca () { return Rang; }
private:
    // svojstva
    int Rang;
};
```


- Međutim, sa prethodnim konceptom povezani su problemi :
 - manje efikasane osnovne operacije - poziv C++ virtualne funkcije je sporiji od običnog poziva funkcije: koristi se `vtable`, pa se funkcija poziva indirektno
 - nespretnost za korisnike - da bi se iskoristila polimorfna svojstva objekta, moraju se koristiti reference i pokazivači
- ```
void PosaljiSvjezePodatke(cTelemetrijskiPodaci d, ...
// ako se predaju cElektricniPodaci -> slicing!, pa treba:
void PosaljiSvjezePodatke(cTelemetrijskiPodaci &d,
 ...
```
- slabije ućahurivanje - virtualne funkcije se mogu prekriti i tada se može pristupati članovima osnovne klase koji su deklarirani kao `protected`

# Apstraktni tipovi

- Najjednostavniji način da se razdvoji interfejs i ugradnja

```
class cSet {
public:
 virtual ~cSet();

 virtual void Insert(void *) = 0;
 virtual void Remove(void *) = 0;

 virtual void *First() = 0;
 virtual void *Next() = 0;
};

class cList_set : public cSet, private cList {
};

class cVector_set : public cSet, private cVector {
};
```

- definira interfejs prema klasi `cSet` sa ugrađenom mogućnošću iteracije preko elemenata

```

void f (cSet &s) {
 for (void *p = s.First(); p; p = p.Next()) {
 // radi nešto
 }
}

```

- apstraktna klasa osigurava zajednički interfejs prema različitim ugradnjama (možemo koristiti `cSet` bez da znamo koja je vrsta ugradnje iskorištena)

```

cList_set sl;
cVector_set v(100);

```

```

void g() {
 f(sl);
 f(v);
}

```

## ● C++ Set

- apstraktni tip:
  - definira jedan koncept na način koja dopušta da u programu postoji više različitih ugradnji
  - osigurava razumnu efikasanost prilikom izvođenja i zauzimanja prostora, preko korištenja virtualnih funkcija
  - dopušta svakoj ugradnji minimalnu ovisnost o drugim klasama
  - shvatljiv je sam po sebi
  
- različita koncepcija od konkretnih tipova
  - apstraktni tipovi su predviđeni za "općenitiji" pristup
  - konkretni tipovi su efikasniji
  - pogrešno je pomoću jednog pokušavati dostići prednost drugog načina
  
- apstraktni tipovi su često predviđeni za daljnje nasljeđivanje (najčešće da se dodaju detalji ugradnje)

- zašto nismo izveli `cList` i `cVector` iz klasa `cSet`, da bi uštedjeli uvođenje klasa `cList_set` i `cVector_set` ?
  - ponovno korištenje - potrebno je iskoristiti već definirane tipove (poput `cVector` i `cList`) dajući im novi interfejs

# Čvorne klase

- Čvorna klasa, oblikovana kao sastavni dio hijerarhije, oslanja se na svoju osnovnu klasu da joj osigura određene usluge (*services*) (poziva funkcije definirane u osnovnoj klasi)
  - također, čvorna klasa ne implementira samo funkcije definirane u interfejsu svoje osnovne klase, već definira i nove

```
class cAuto : public cVozilo {
public:
 cAuto(int Putnika, int Velicina, int Tezina, int fc)
 : cVozilo(Putnika, Velicina, Tezina), KolicinaGoriva(fc) { }
 // prekrivanje virtualnih funkcija iz Vozila
 void Okreni(int Smjer);
 // dodavanje specifičnih funkcija za Auto
 virtual DodajGorivo(int Kolicina);
private:
 int KolicinaGoriva;
};
```

- važan je konstruktor kroz kojeg programer specificira osnovna svojstva, i virtualne funkcije koje dozvoljavaju rutinama za simulaciju da manipuliraju objektima klase `cAuto` bez poznavanja točnog tipa

- Čvorna klasa obično treba konstruktor, i to često netrivialni (razlika prema apstraktnim klasama)
- za postupke nad klasom `cAuto` često će se koristiti funkcije iz klase `cVozilo`

```
bool Most::MozePrijeci(const cVozilo& R) {
 if (MaxTezina < R.Tezina())
 return false;
 // ...
}
```

- sama klasa `cAuto` je dobar kandidat za daljnje nasljeđivanje

```
class cHitnaPomoc : public cAuto, public cEmergency {
public:
 cHitnaPomoc();
 // prekrivanje relevantnih funkcija iz klase Auto
 void Okreni(int Smjer);
 // prekrivanje relevantnih funkcija iz klase Emergency
 virtual Posalji(const Lokacija&);
 // dodavanje funkcije specifičnih za HitnuPomoc
 virtual int KapacitetPacijenata();
};
```

- Čvorna klasa se:
  - oslanja na bazni klasa i za svoju ugradnju i za osiguravanje usluga svojim korisnicima
  - osigurava širi interfejs
  - ovisi o svim svojim (direktnim i indirektnim) osnovnim klasama
  - može se shvatiti samo u kontekstu svojih osnovnih klasa
  - može se koristiti za daljnje nasljeđivanje
  - može se koristiti za stvaranje (instanciranje) objekata
- Čvorne klase ponekad ne zadovoljavaju sve navedene točke



# Akcije

- Akcije se tipično ostvaruju funkcijom
- Ponekad je korisno akciju koja se treba nakratko odgoditi, koja treba biti poslana drugdje, zahtijeva vlastite podatke, mora se kombinirati sa ostalim akcijama itd., načiniti kao klasa koja može izvršiti željenu akciju i osigurati druge usluge
- Najjednostavniji oblik:

```
class cAkcija {
public:
 virtual int Nacini() = 0;
 virtual ~cAkcija() {}
};
```

● C++  Action

# Klase interfejsi

- Prilagođava pojavnost neke usluge lokalnim potrebama
  - Primjer : indeksi klasičnog vektora definirani su u rasponu  $[0, n-1]$ . Korisnici koji žele koristiti druge raspone, moraju prilagoditi uporabu

```
void f() {
 cVector v(10); // vektor sa 10 int elemenata, indeksi [0,9]
 for (int i=1; i<=10; i++) // ponašamo se kao da je raspon [1:10]
 v[i-1] = 7; // ali moramo prilagoditi stvarni indeks
}
```

```
class cVectorDG : public cVector {
 int DG;
public:
 cVectorDG(int Donja, int Gornja) : Vector(Gornja-Donja+1)
 { DG=Donja; }
 int &operator[](int i)
 { return cVector::operator[] (i-DG); }
 int Donja() { return DG; }
 int Gornja() { return DG + GetMaxElem() - 1; }
};
```

## ■ način korištenja

```
void g() {
 cVectorDG V(1,10);

 for (int i=1; i<=10; i++)
 V[i] = 7;
}
```

- u usporedbi sa rješenjem bez prilagođavanja interfejsa, nema viška kôda, a jednostavnije je za čitati i predstavlja elegantnije rješenje
- klase interfejsi su uglavnom mali i (po definiciji) čine malo
- pojavljuju se kada softver pisan po različitim tradicijama i konvencijama mora surađivati (npr. za osiguravanje C++ interfejsi prema ne-C++ kodu ili za razdvajanje kôda programa od detalja programske knjižnice)
- druga važna uloga klasa interfejsa je da omoguće provjeru ulaznih parametara za postojeće klase

● C++  Interface

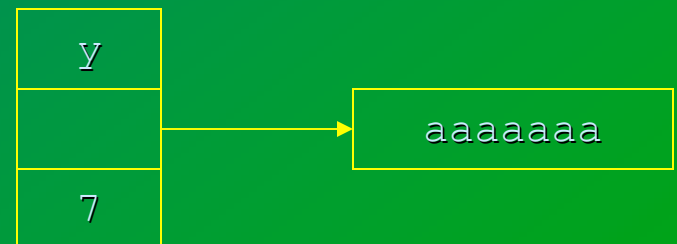
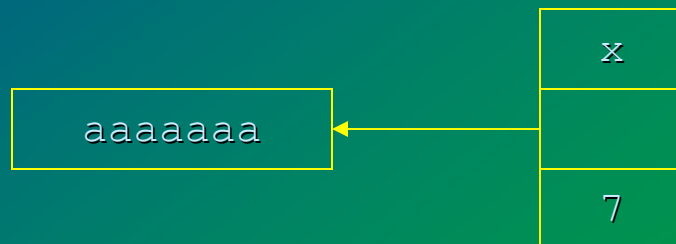
# Oznake (*handles*)

- Podjela jednog objekta na dva dijela :
  - oznaka koja osigurava korisničko interfejs
  - reprezentacija koja sadrži većinu ili sva svojstva objekta
  - veza između oznake i reprezentacije je obično pokazivač u klasi oznake
  - iako klasa oznake obično sadrži više podataka od jednog pokazivača, sadržaj klase oznake je stabilan čak i kad se reprezentacija mijenja

## ■ Primjer: cNiz

C++  Niz

```
class cNiz {
public:
 ... // interfejs
 cNiz & operator=(const char P[]);
 cNiz & operator=(const cNiz &P);
private:
 char *S;
 int BrojZnakova;
};
...
cNiz x, y;
x = "aaaaaaa";
y = x;
```

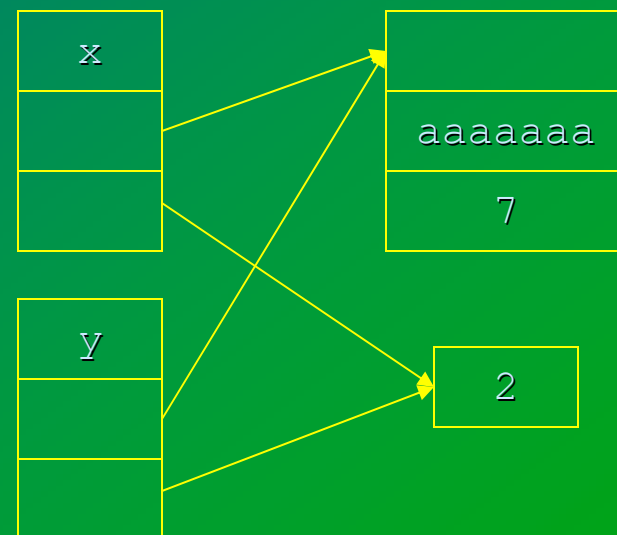


## ■ klasa oznake s brojačem referenci

C++  NizHandle

```
class cNizHandle {
public:
 ... // interfejs
 cNizHandle & operator=(const char P[]);
 cNizHandle & operator=(const cNizHandle &P);
private:
 cNiz *Niz;
 int *BrojRef;
};

...
cNizHandle x, y;
x = "aaaaaaa";
y = x;
```



## ■ Općenito

```
template <class _Type> class cHandle {
private:
 _Type *Rep;
 int *BrojRef;

public:
 cHandle(_Type *p) : Rep(p), BrojRef(new int(1)) { }
 cHandle(const cHandle &r) :
 Rep(r.Rep), BrojRef(r.BrojRef)
 { (*BrojRef)++; }

 _Type *operator->() { return Rep; }
 cHandle & operator=(const cHandle &r);

 ~cHandle();
};
```

- ponekad je korisno izdvojiti pokazivač na reprezentaciju i koristiti ga direktno (ukoliko se treba predati nekoj funkciji koja ne zna raditi sa klasom oznake)
  - u tom slučaju je bitno da ta funkcija ne uništi objekt koji joj se predaje i da ne pohrani pokazivač za kasniju upotrebu
- korisna je mogućnost i dodjeljivanja nove reprezentacije oznaci

```
template<class _Type> class cHandle {
 // ...

 _Type *GetRep() { return Rep; }

 void DodijeliNovuRep(_Type *p) {
 if (p != Rep) {
 if (--*BrojRef == 0) {
 delete Rep;
 *BrojRef = 1;
 } else
 BrojRef = new int(1);
 Rep = p;
 }
 }
};
```



- ovakva klasa `cHandle` obično se ne nasljeđuje (radi se o konkretnom tipu bez virtualnih funkcija)
- klasa oznake koja osigurava interfejs gotovo identično interfejsu klasa čija je to oznaka se često naziva *proxy*
- prekrivanje operatora → omogućava klasi oznake da obavi neki posao prilikom svakog pristupa objektu

```
template<class _Type> class cXhandle {
 _Type *Rep;
 int BrojPristupa;
public:
 _Type *operator->() { BrojPristupa++; return Rep; }
};
```

- moguće je obavljati neki posao prije i poslije pristupa (primjerice zaključavanje zapisa prilikom pristupa ili izmjene)

# Kosturi programa

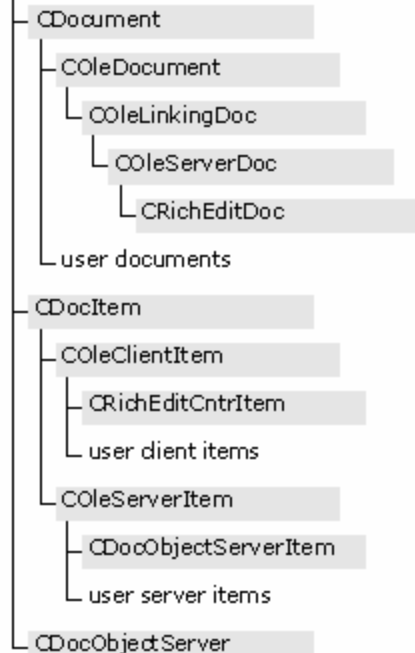
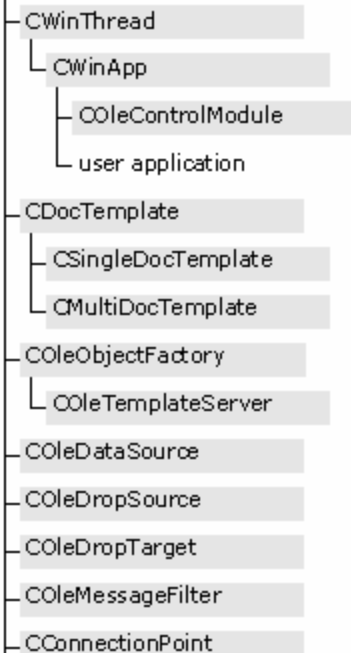
- prilikom izgradnje kostura programa oblikuje se i piše kôd koji pruža okvir unutar kojega programer smješta kôd specifičan za aplikaciju
- klase koje čine takav okvir imaju predimenzionirani interfejsi (*fat interfaces*)
- približno predstavljaju kompletnu aplikaciju, ali ne čine ništa, već stvarne akcije specificira programer
- primjer : MFC (Microsoft Foundation Classes) application framework
  - document/view arhitektura

## Microsoft Foundation Class Library Version 4.21

### CObject

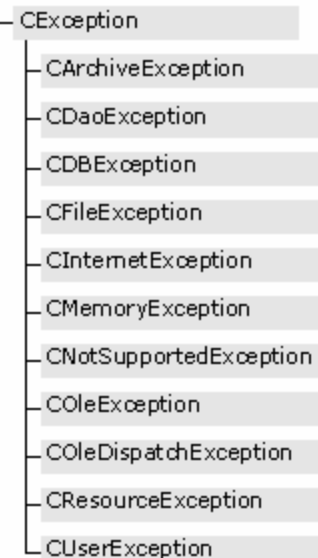
#### Application Architecture

##### CcmdTarget



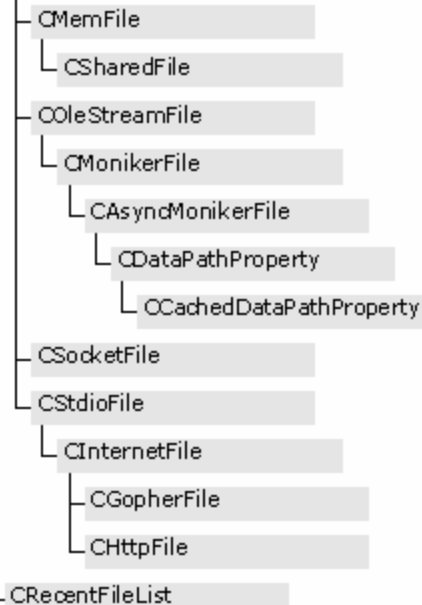
##### user objects

#### Exceptions



#### File Services

##### CFile



## Window Support

### CWnd

#### Frame Windows

- CFrameWnd
  - CMDIChildWnd
    - user MDI windows
  - CMDIFrameWnd
    - user MDI workspaces
  - CMiniFrameWnd
    - user SDI windows
  - COleIPFrameWnd

CSplitterWnd

#### Control Bars

- CControlBar
  - CDialogBar
  - COleResizeBar
  - CStatusBar
  - CToolBar

#### Property Sheets

- CPropertySheet

#### Dialog Boxes

- CDialog
  - CCommonDialog
    - CColorDialog
    - CFileDialog
    - CFindReplaceDialog
    - CFontDialog
    - COleDialog
      - COleBusyDialog
      - COleChangeIconDialog
      - COleChangeSourceDialog
      - COleConvertDialog
      - COleInsertDialog
      - COleLinksDialog
        - COleUpdateDialog
      - COlePasteSpecialDialog
      - COlePropertiesDialog
    - CPageSetupDialog
    - CPrintDialog
  - COlePropertyPage
  - CPropertyPage
  - user dialog boxes

#### Views

- CView
  - COtrlView
    - CEditView
    - QListView
    - CRichEditView
    - CTreeView
  - CScrollView
    - user scroll views
  - CFormView
    - user form views
  - CDaoRecordView
    - CRecordView
      - user record views

#### Controls

- CAnimateCtrl
- CButton
  - CBitmapButton
- CComboBox
- CEdit
- CHeaderCtrl
- CHotKeyCtrl
- QListBox
  - CCheckListBox
  - CDragListBox
- QListCtrl
- COleControl
- CProgressCtrl
- CRichEditCtrl
- CScrollBar
- CSliderCtrl
- CSpinButtonCtrl
- CStatic
- CStatusBarCtrl
- CTabCtrl
- CToolBarCtrl
- CToolTipCtrl
- CTreeCtrl

# Notacija

- Dijagrami koji predstavljaju pogled na modele sistema
- Statički prikaz
  - **Dijagram klase** - koje klase postoje i kako su povezane
  - **Dijagram objekata** - koji mehanizmi se koriste za reguliranje međudjelovanja objekata
  - **Dijagram modula** - gdje su deklarirane i ugrađene klase
  - **Dijagram procesa** - kojem procesoru bi se trebao dodijeliti koji proces
- problem opisa dinamičkih događaja u sistemu na komadu papira
- Dinamički prikaz
  - **Dijagram prijelaza stanja** - opisuje ponašanje sistema prilikom različitih događaja
  - **Dijagram interakcija** - opisuje redoslijed izvođenja

# Dijagram klasa

- Bitni elementi :
  - klase
  - odnosi među klasama
- Oznaka klasa



- atributi označavaju dio složenog objekta
- označavanje atributa unutar klasa:
  - A                      samo ime atributa
  - : C                    samo klasa atributa
  - A : C                ime i klasa atributa
  - A : C = E            ime, klasa i pretpostavljena vrijednost (izraz)

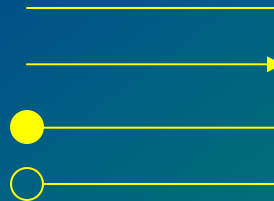
- postupak označava neku uslugu (*service*) koju pruža klasa
- označavanje postupaka :
  - N() - samo ime postupka
  - R N(Arg) - povratni tip, ime i formalni argumenti postupka
- primjer klase cSkladište



## ■ Odnosi među klasama

### ● oznake odnosa

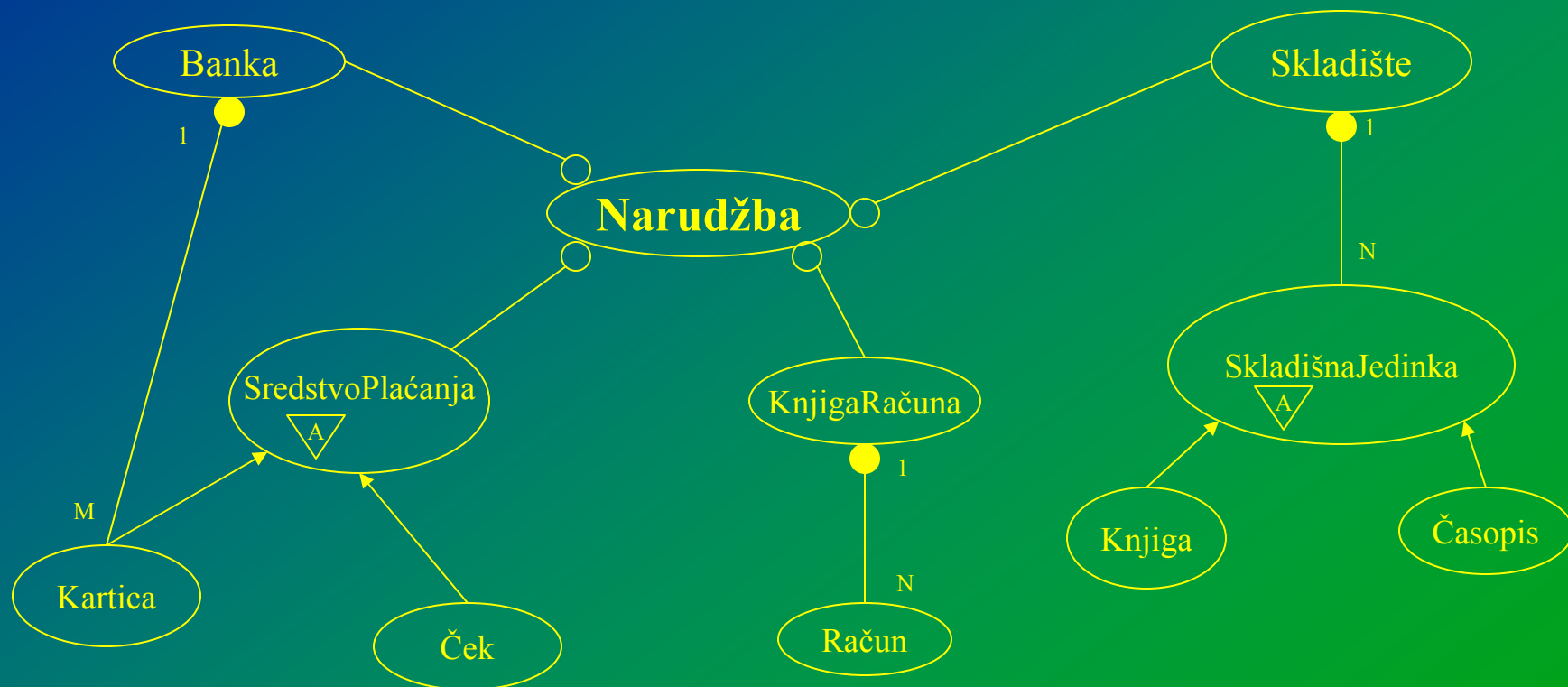
- asocijacija
- nasljeđivanje
- sadržavanje
- korištenje



- oznaka asocijacije povezuje dvije klase i označava semantičku vezu među klasama
- klasa može imati asocijaciju sa samim sobom (refleksivna asocijacija)
- asocijaciji se može pridružiti kardinalitet
  - 1                      točno jedan
  - N                      neograničen broj
  - 0 .. N                nijedan ili više
  - 1 .. N                jedan ili više
  - 0 .. 1                nijedan ili jedan
  - 3 .. 7                određeni raspon
  - 1 .. 3, 7              određeni raspon ili točan broj



- Odnosi *nasljeđivanje*, *sadržavanje* i *korištenje* predstavljaju profinjene asocijacije (tokom razvoja, najprije se ustanovljava semantička veza, a zatim se određuje koji od gornja tri odnosa najbolje odgovara)
- Primjer :



Oznaka apstraktnog klasa

## ■ *class utilities*

- neki programski jezici dopuštaju miješanje proceduralnog programiranja sa objektno-orijentiranim (npr. C++)
- postoje funkcije u programu koje nisu članovi niti jednog klasa (slobodni potprogrami)
- *class utilities* se pojavljuju u dvije varijante:
  - slobodni potprogrami
  - klase koji imaju varijable i postupke zajedničke za sve objekte nekog klasa (`static` varijable i članske funkcije u C++)



## ■ Ugniježdene klase (nested class)

- deklaracije klasa se mogu ugniježdziti
- Ako je klasa `Račun` korištena samo od klasa `KnjigaRačuna`, njegova deklaracija se može ugniježdziti unutar deklaracije klasa `KnjigaRačuna`

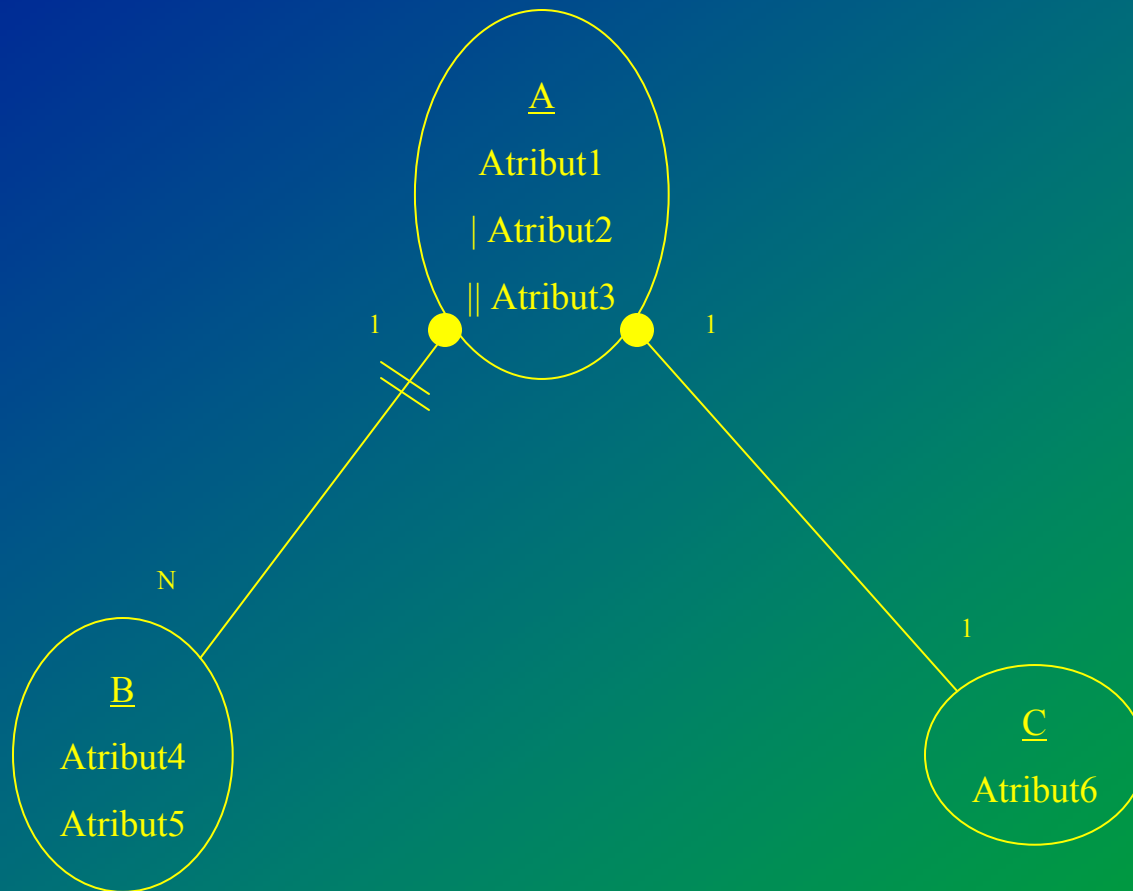
```
class cKnjigaRacuna {
 ...
 class cRacun {
 ...
 };
 ...
};
```



- u tom slučaju, kvalificirano ime klase postaje `cKnjigaRačuna::cRacun`
- ugniježdjena klasa se ne može naslijediti

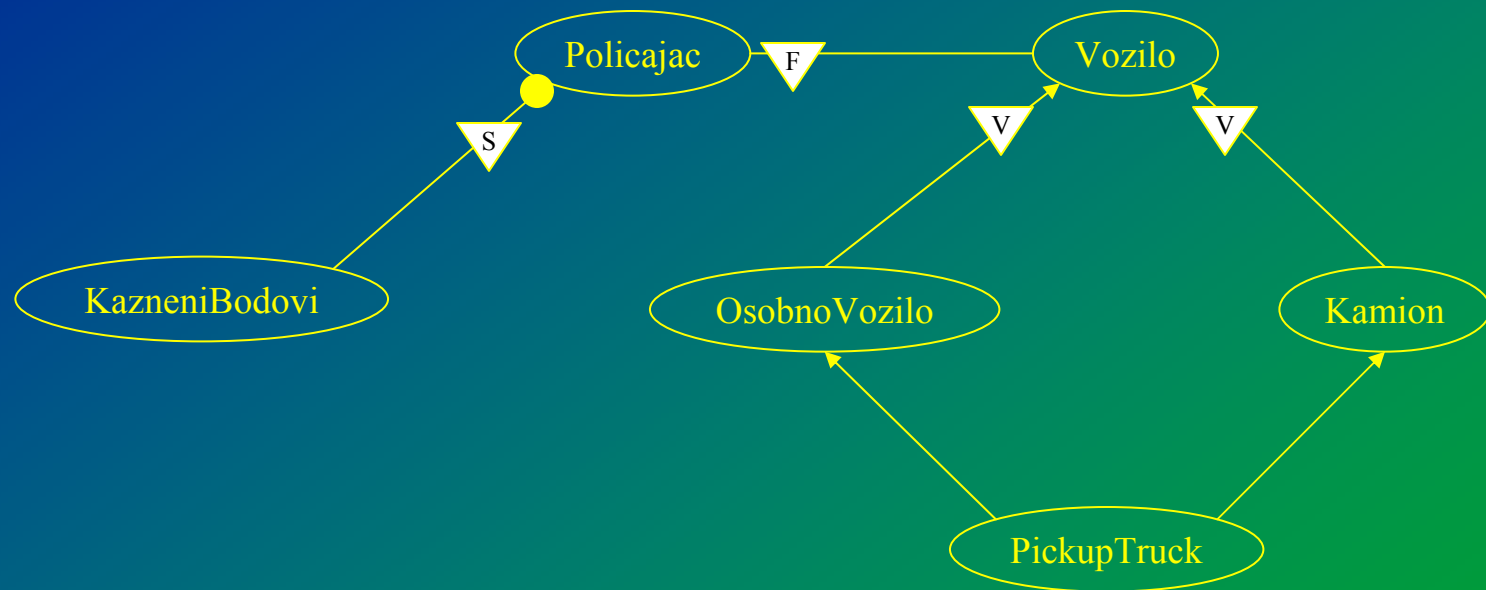
# Kontrola pristupa

- većina OO-jezika omogućava neku vrstu kontrole pristupa članovima klase
  - bez oznake `public` pristup
  - `|` `protected` pristup (dozvoljen klasi, izvedenim klasama i `friend` funkcijama)
  - `||` `private` pristup (dozvoljen klasi i `friend` funkcijama)
  - `|||` implementacijski pristup (dio implementacije klase koji nije pristupačan čak ni `friend` funkcijama)
- Pri nasljeđivanju:
  - `public`: `public` i `protected` članovi osnovnog klase postaju redom `public` i `protected` članovi izvedene klase
  - `protected`: `public` i `protected` članovi osnovnog klase postaju `protected` članovi izvedene klase
  - `private`: `public` i `protected` članovi osnovnog klase postaju `private` članovi izvedene klase



# Svojstva

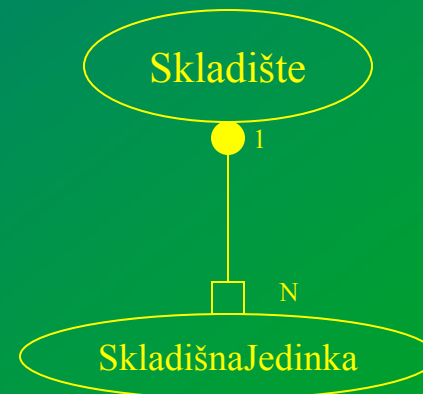
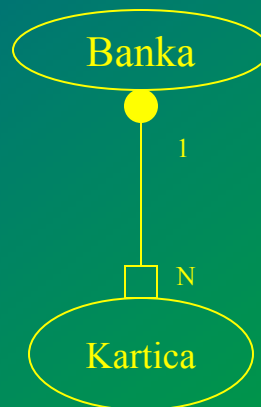
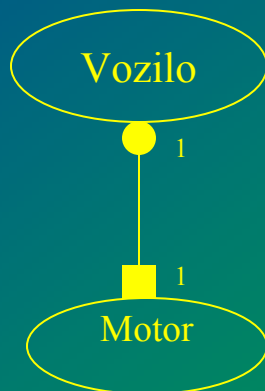
- za neke jezike, određeni odnosi među klasama su od iznimnog značaja, te zahtijevaju uvođenje posebne notacije
- u C++ su to tri svojstva:
  - `static` - označavanje članske varijable ili funkcije u klasi
  - `friend` - označavanje klase koja daje pravo drugoj klasi da pristupa njegovim ne-`public` dijelovima
  - `virtual` - označavanje dijeljene osnovne klase kod višestrukog nasljeđivanja
- označavanje slično kao za apstraktne klase
  - `static` 
  - `virtual` 
  - `friend` 



```
class cVozilo {
 friend class cPolicajac; ...
class cOsobnoVozilo : public virtual cVozilo { ...
class cKamion : public virtual cVozilo { ...
class cPickupTruck : public cOsobnoVozilo, public cKamion { ...
```

# Označavanje sadržavanja

- fizičko sadržavanje igra ulogu kod konstrukcije odnosno destrukcije objekta
- sadržavanje može biti
  - po vrijednosti
  - po referenci
- ukoliko se drugi dio oznake za sadržavanje ne definira, smatra se da je način sadržavanja nespecificiran





# Specifikacije

- Specifikacija je negrafički oblik koji se koristi da osigura potpunu definiciju entiteta u notaciji (klasa, odnosa, postupaka)
- posebno je važno izložiti specifikacije elemenata koje nemaju svoje mjesto u dijagramima
- svaka specifikacija mora imati :
  - *Ime* : identifikator
  - *Definicija* : tekst
- specifikacije klasa :
  - *Odgovornosti* : tekst
  - *Atributi* : popis atributa
  - *Postupci* : popis postupaka
  - *Ograničenja* : popis ograničenja
- moguća dodatna specifikacija (za klasu koja ima takav opis)
  - *Dijagram stanja* : referenca na dijagram stanja

- napredne specifikacije bez odgovarajućih elemenata u dijagramima (odgovaraju nekim funkcionalnim aspektima)
  - *Postojanost* : tranzijentno | postoјano
  - *Istodobnost* :
    - sekvencijalni (OK kod jednonitnog (jednodretvenog) izvođenja)
    - čuvani (klase surađuju na uzajamnom isključivanju)
    - sinkroni (klasa se sam brine o uzajamnom isključivanju)
    - aktivni (klasa ima vlastitu nit (dretvu))
  - *Zauzeće prostora* : izraz

- specifikacije postupaka - definiraju se specifikacije za svaki slobodni potprogram ili člansku funkciju
  - *Povratna klasa*      referenca na klasu
  - *Argumenti*      lista formalnih argumenata
  - *Kvalifikacija*      u C++ npr. `static`, `virtual`, `const`
  - *Kontrola pristupa*    `public` | `private` | `protected` | ugradnja
- sljedeće stavke prikazuju semantiku postupaka :
  - *Ulaz*      tekst | referenca na kôd | referenca na objektni dijagram
  - *Semantika*    tekst | referenca na kôd | referenca na objektni dijagram
  - *Izlaz*      tekst | referenca na kôd | referenca na objektni dijagram
  - *Izuzeci*      popis izuzetaka
- dodatne napredne stavke za definiranje odgovarajućih funkcionalnih aspekata postupaka :
  - *Istodobnost*      sekvencijalna | čuvana | sinkrona
  - *Zauzeće prostora*      izraz
  - *Procjena trajanja*      izraz

# Razvoj aplikacije za praćenje vremenskih uvjeta

## ■ Zahtjevi na aplikaciju :

### ● Mjerenje sljedećih veličina :

- brzine i smjera vjetra
- temperature
- tlaka zraka
- vlažnosti zraka

### ● Pružanje informacija o :

- kretanju temperature
- kretanju tlaka zraka

- sistem mora imati mjerenje vremena kako bi se mogla dobiti informacija o najnižim i najvišim temperaturama u zadnja 24 sata
- sistem mora imati Ekran koji prikazuje sve mjerene i izračunate vrijednosti te trenutni datum i vrijeme
- sistem mora dopustiti korisniku kalibriranje senzora prema poznatim vrijednostima i postavljanje trenutnog datuma i vremena

## ■ Hardverske pretpostavke

- koristi se PC kompatibilno računalo, MS Windows
- vrijeme i datum se saznaju sa sata na ploči, dostupnog preko funkcija operacijskog sistema
- temperatura, tlak zraka i vlažnost se mjere preko udaljenih senzora, a izračunate vrijednosti su dostupne preko I/O memorijskog mapiranja
- korisnički ulaz je preko tipkovnice
- Ekran je LCD uređaj
- u sistemu postoji *timer*

# Vrijeme i datum

- učajuruje funkcije za rad s datumom i vremenom
- specifikacija :

Ime :

DatumIVrijeme

Odgovornosti :

Vodi računa o trenutnom vremenu i datumu

Postupci :

GetNeformatiranoVrijeme

SetVrijeme

SetDatum

GetVrijeme

GetDatum

# Senzor temperature

- učajuruje ponašanje senzora temperature
- specifikacija :

Ime :

SenzorTemperature

Odgovornost :

Prati trenutnu vrijednost temperature

Postupci :

GetTemperatura

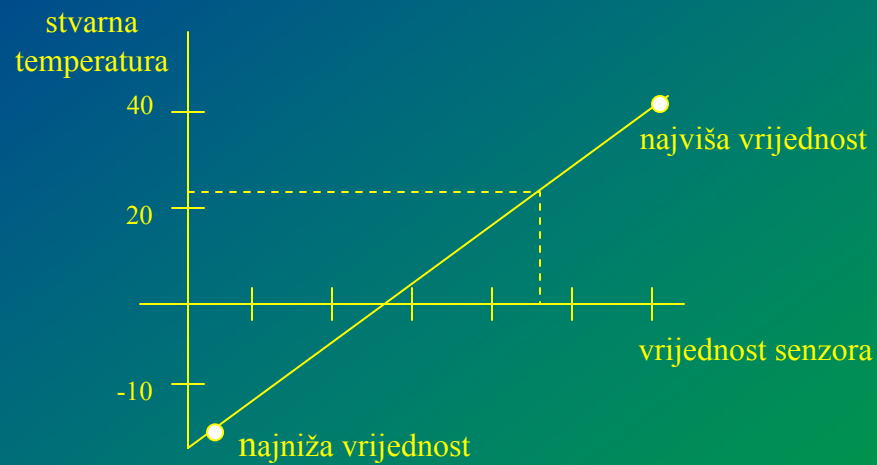
SetNajnižuTemperaturu

SetNajvišuTemperaturu

Atributi :

Temperatura

- postupci za postavljanje najniže i najviše temperature su potrebni za kalibriranje senzora (senzor vraća bezdimenzionalnu vrijednost koja se onda linearno interpolira između najviše i najniže vrijednosti)



Kalibracija i očitavanje senzora temperature



# Senzor tlaka zraka

- **specifikacija :**

Ime :

SenzorTlaka

Odgovornost :

Prati trenutnu vrijednost tlaka zraka

Postupci :

GetTlak

SetNajnižiTlak

SetNajvišiTlak

Atributi :

Tlak

- kod mjerenja temperature i tlaka zraka još jedan bitan podatak bi mogao biti trend kretanja (u kojem smjeru se mijenja odgovarajuća veličina)
- i za `SenzorTemperature` i za `SenzorTlaka` možemo izraziti trend kretanja kao realni broj koji predstavlja nagib krivulje odgovarajuće veličine (derivaciju po vremenu)

# klasa TrendSenzor

- specifikacija :

Odgovornost :

Vraća trend kretanja tlaka ili temperature kao nagib pravca koji odgovara prethodno izmjeranim vrijednostima

Postupci :

Trend

Atributi :

PrethodnaVrijednost

VrijemePrethodneVrijednosti

- budući da je takvo ponašanje zajedničko za senzore temperature i tlaka zraka, možemo ga učahuriti u nadklasa koji će oba klasa naslijediti
- uvodimo klasa TrendSenzor

# Senzor vlažnosti zraka

- apstrakciju senzora vlažnosti zraka možemo izraziti pomoću slijedeće specifikacije :

Ime:

SenzorVlažnosti

Odgovornost :

Prati trenutnu vlažnost zraka, izraženu u postotcima (0-100%)

Postupci:

GetVlažnost

SetNajnižuVlažnost

SetNajvišuVlažnost

Atributi

Vlažnost

## klasa PamtećiSenzor

- za sva tri prethodno navedena senzora imamo zajednički zahtjev da se pamte najniže i najviše vrijednosti u zadnja 24 sata
- možemo to definirati zajedničkom specifikacijom :

Odgovornosti :

Prijavljuje najniže i najviše vrijednosti u zadnja 24 sata

Postupci :

GetMinimum

GetMaksimum

GetVrijemeMinimuma

GetVrijemeMaksimuma

Atributi:

Minimum, Maksimum

VrijemeMinimuma, VrijemeMaksimuma

- ovaj će klasa biti zajednički nadklasa za tri navedena klasa senzora

# Senzor za očitavanje brzine vjetra

- specifikacija :

Ime :

SenzorBrzineVjetra

Odgovornosti :

Prati trenutnu brzinu vjetra

Postupci :

GetBrzina

SetNajnižuBrzinu

SetNajvišuBrzinu

Atributi :

Brzina

- brzina vjetra se izračunava iz broja okretaja vjetrokaza, ali je to implementacijski detalj koji se ne tiče korisnika klasa

- sva četiri prethodna klasa imaju zajedničku karakteristiku da pravu vrijednost veličine izračunavaju interpolacijom preko dvije poznate vrijednosti, postavljene kalibracijom
- da se funkcionalnost kalibracije i interpolacije ne bi ponavljala u sva četiri klasa, definirat će se jedan nadklasa. Kalibracija će se obaviti u konstruktoru.
- specifikacija :

Ime :

KalibrirajućiSenzor

Odgovornosti :

Osigurava kalibraciju i linearnu interpolaciju između dvije poznate točke

Postupci :

SetVrijednost

Atributi:

Minimum, Maksimum

VrijednostMinimума, VrijednostMaksimuma

- **klasa** KalibrirajućiSenzor će biti neposredni nadklasa klasi PamtećiSenzor

# Senzor za očitavanje smjera vjetra

- ovaj klasa se razlikuje od prethodnih jer nema kalibraciju, niti potrebu za pamćenjem prethodnih vrijednosti
- specifikacija :

Ime :

`SenzorSmjeraVjetra`

Odgovornosti :

`Prati trenutni smjer vjetra`

Postupci :

`GetSmjer`

Atributi

`Smjer`

# Senzor

- Zajedničko svim senzorima: adresa s koje se uzima podatak, vrijednost posljednjeg očitavanja, vrijeme posljednjeg očitavanja
- specifikacija :

Ime :

Senzor

Odgovornost :

Očitava vrijednost s neke adrese i pamti kad se to dogodilo

Postupci :

SetVrijednost

GetVrijednost

GetVrijemeMjerenja

Atributi :

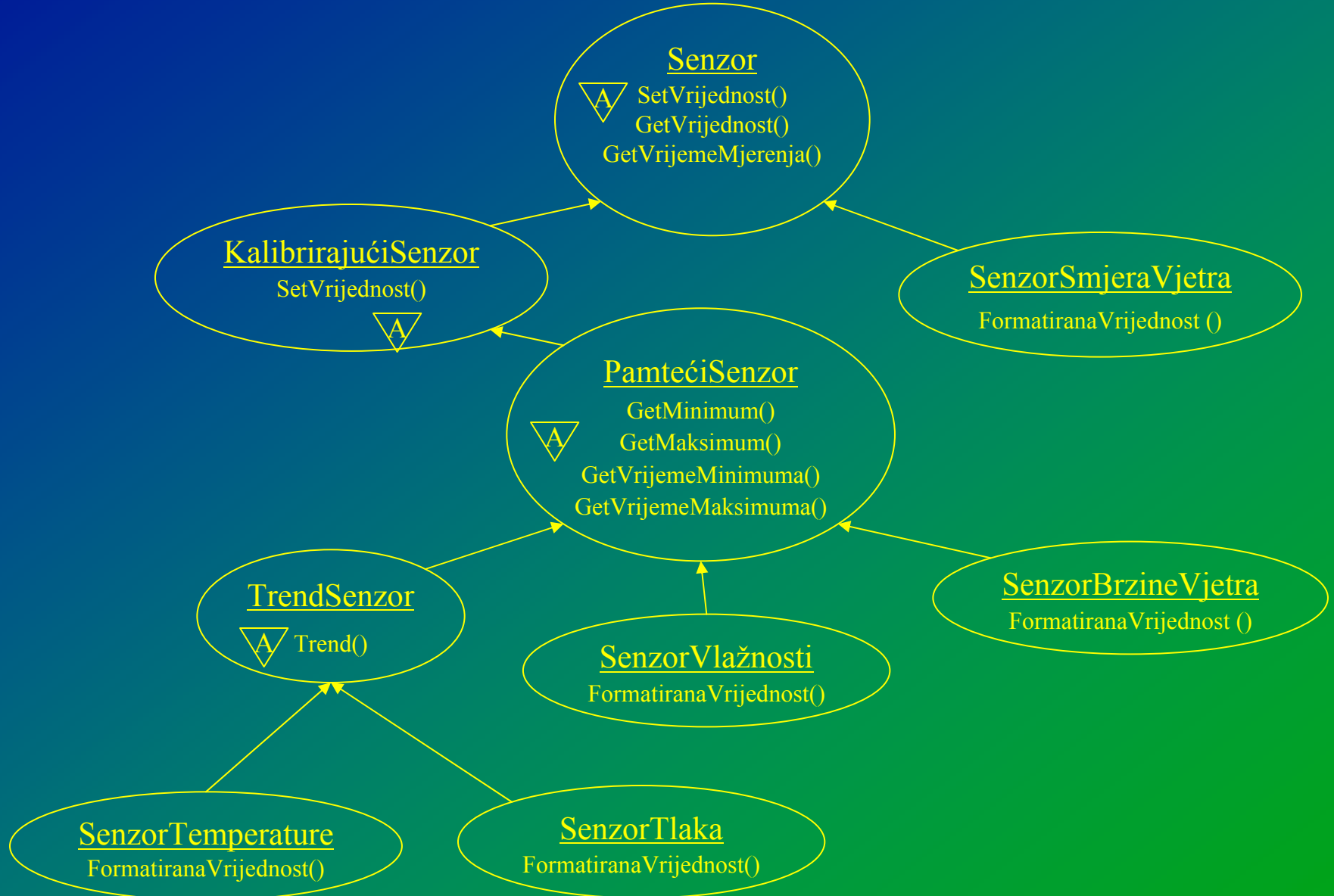
Adresa

Vrijednost

VrijemeMjerenja



- cjelokupna hijerarhija ima sljedeći izgled



## ■ izgled ispisa na LCD Ekranu

|               |            |
|---------------|------------|
| Datum         | 01.12.1998 |
| Vrijeme       | 13:37      |
| Temperatura   | 21°C       |
| Tlak          | 1023 hP    |
| Vlažnost      | 63%        |
| Smjer vjetra  | NW         |
| Brzina vjetra | 25 km/h    |

## ■ specifikacija:

Ime :

LCDUređaj

Odgovornosti :

Osigurava usluge za prikazivanje grafičkih elemenata

Postupci :

SetVelicinuTeksta

SetStilTeksta

IspisiTekst

# Timer

- pretpostavljamo da u sistemu postoji *timer*, koji okida prekid u zadanom intervalu, i pri tome poziva funkciju za opsluživanje prekida
- napraviti ćemo klasu koji će koristiti `callback` funkciju. Klijent mora osigurati `callback` funkciju i pobrinuti se da se obavi do sljedećeg prekida od timera
- specifikacija :

Ime :

Timer

Odgovornosti :

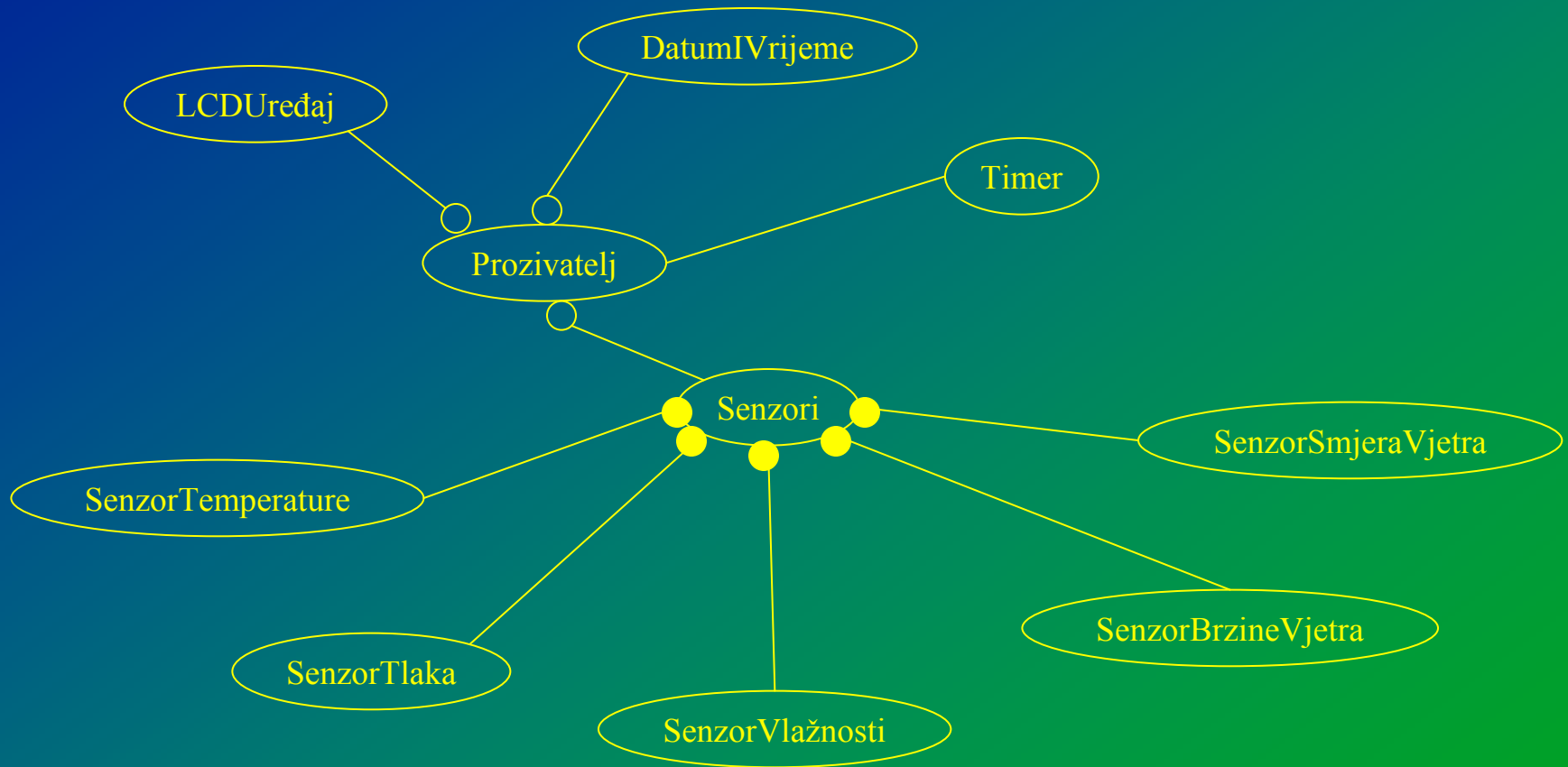
Poziva `callback` funkciju

# Scenariji događaja

- nakon opisa sistema, analiziramo nekoliko različitih scenarija korištenja sistema
- primarne funkcije sistema:
  - uključivanje sistema u rad
  - kalibracija senzora
  - postavljanje datuma i vremena
  - praćenje vremenskih uvjeta (temperatura, tlak, vlažnost, vjetar)
  - praćenje izračunatih vrijednosti (trend kretanja temperature i tlaka)
  - prikazivanje najnižih i najviših vrijednosti
- moramo se odlučiti koliko često ćemo očitavati vrijednosti vremenskih varijabli
  - svakih 0.5 sekundi                      smjer vjetra
  - svake sekunde                              brzina vjetra
  - svakih 5 minuta                           temperatura, vlažnost, tlak

- budući da smo već definirali da klase koji predstavljaju senzore nemaju nikakvu odgovornost za rad sa vremenski tempiranim događajima, moramo ubaciti vanjskog *agenta* koji će surađivati sa senzorima u provedbi mjerenja
- agent će senzore prozivati u različitim intervalima (navedenim prije)
- prozivanjem pojedinog senzora, umjesto dopuštanja svakom senzoru da ima svoju nit izvođenja, pojednostavnjuje se izvođenje programa (i postaje predvidljivije)
- radi jednostavnosti uvodimo klasa `Senzori` koji sadrži skup senzora
- uvodimo klasa `Prozivatelj`
  - Ime :  
    `Prozivatelj`
  - Odgovornosti :  
    Proziva senzore i prikazuje rezultat na Ekranu
  - Postupci :  
    `Prozovi`

# Dijagram klasa



# Ugradnja

## ■ Senzor

```
class cSenzor {
public:
 cSenzor (int inAdresa) { Adresa = inAdresa; }
 virtual FormatiranaVrijednost () = 0;
 float GetVrijednost () { return Vrijednost; }
 time_t GetVrijemeMjerenja () { return VrijemeMjerenja; }
 void SetVrijednost () {
 Vrijednost = *((float *) Adresa);
 VrijemeMjerenja = DatumIVrijeme.NeformatiranoVrijeme();
 }
private:
 float Vrijednost;
 time_t VrijemeMjerenja;
 int Adresa;
}
```

## ■ KalibrirajuciSenzor

```
class cKalibrirajuciSenzor {
public:
 cKalibrirajuciSenzor (int Adresa,
 float inMinimum, float inVrijednostMinimuma,
 float inMaksimum, float inVrijednostMaksimuma)
 : cSenzor (Adresa) {
 Minimum = inMinimum;
 ...
 }
 void SetVrijednost () {
 cSenzor::SetVrijednost ();
 Vrijednost = (Vrijednost - Minimum) / (Maksimum - Minimum) *
 (VrijednostMaksimuma - VrijednostMaksimuma) +
 (VrijednostMinimuma)
 }
private:
 float Minimum, VrijednostMinimuma, Maksimum, VrijednostMaksimuma;
}
```



## ■ PamteciSenzor

```
class cPamteciSenzor : public cKalibrirajuciSenzor {
public:
 float GetMinimum() { return Minimum; }
 float GetMaksimum() { return Maksimum; };
 time_t GetVrijemeMinimuma();
 time_t GetVrijemeMaksimuma();
 void SetVrijednost () {
 KalibrirajuciSenzor::SetVrijednost();
 if (Vrijednost < Minimum) {
 Minimum = Vrijednost;
 VrijemeMinimuma = VrijemeMjerenja;
 }
 if (Vrijednost > Maksimum) {
 Maksimum = Vrijednost;
 VrijemeMaksimuma = VrijemeMjerenja;
 }
 }
private:
 float Minimum, Maksimum;
 time_t VrijemeMinimuma, VrijemeMaksimuma;
}
```

## ■ TrendSenzor

```
class cTrendSenzor : public cPamteciSenzor {
public:
 float Trend () {
 return
 (Vrijednost - PrethodnaVrijednost) /
 (VrijemeMjerenja - PrethodnoVrijeme);
 }
 void SetVrijednost () {
 PrethodnaVrijednost = Vrijednost;
 PrethodnoVrijeme = VrijemeMjerenja;
 PamteciSenzor::SetVrijednost();
 }
private:
 float PrethodnaVrijednost; time_t PrethodnoVrijeme;
}
```

## ■ Stvarni senzori

```
class cSenzorTemperature : public cTrendSenzor {
public:
 char *FormatiranaVrijednost() {
 static char buf[40];
 sprintf (buf, "Temperatura: %5.1f°C", Vrijednost);
 return buf;
 }
}

...
class cSmjerVjetra : public cSenzor {
public:
 char *FormatiranaVrijednost() {
 static char S[16][] = {"N", "NNE", "NE", "ENE", ... };
 sprintf (buf, "Smjer vjetra: %s", S[(int) Vrijednost]);
 return buf;
 }
}
```

## ■ Senzori

```
class cSenzori {
public:
 cSenzori () { BrojSenzora = 0 };
 void Dodaj (cSenzor *P, float Interval) {
 S[BrojSenzora].pSenzor = P;
 S[BrojSenzora++].Brojac = (int) (Interval * 60);
 }
 int GetBrojSenzora () { return BrojSenzora; }
 cSenzor *pSenzor(int i) { return S[i].pSenzor; }
 int Brojac(int i) { return S[i].Brojac; }
private:
 struct {
 cSenzor *pSenzor;
 int Brojac;
 } S[MAXSENZORA];
 int BrojSenzora;
}
```

## ■ Prozivatelj

```
extern cSenzori Senzori, cLCDUredjaj LCDUredjaj,
 cDatumIVrijeme DatumIVrijeme;
class cProzivatelj {
public:
 static void __stdcall Prozovi
 (HWND hWND, UINT uMsg, UINT idEvent, DWORD lngSysTime) {
 int i;
 Brojac++;
 for (i = 0; i < Senzori.BrojSenzora(); i++) {
 if (Brojac % Senzori.Brojac(i) == 0) {
 Senzori.pSenzor(i)->SetVrijednost();
 LCDUredjaj.IspisiTekst (DatumIVrijeme.GetDatum());
 LCDUredjaj.IspisiTekst (DatumIVrijeme.GetVrijeme());
 LCDUredjaj.IspisiTekst
 ((Senzori.pSenzora(i))->FormatiranaVrijednost());
 }
 }
 }
private:
 static int Brojac;
}
```

## ■ Timer

```
// kod zavisen od sistema, npr. za Windows
class cTimer {
public:
 cTimer (float Interval, void (__stdcall *CB) (HWND hwnd...));
 ~cTimer ();
private:
 int Oznaka;
};

cTimer::cTimer (float Interval, void (__stdcall *CB) (HWND hwnd...)) {
 Oznaka = SetTimer(NULL, 0, (int)(Interval/1000), CB);
}

cTimer::~~cTimer () {
 KillTimer(NULL, Oznaka);
}
```

## ■ Glavni program

```
cSenzori Senzori, cLDCUredjaj LCDUredjaj, cDatumIVrijeme DatumIVrijeme;
main () {
 cProzivatelj Prozivatelj;
 Senzori.Dodaj (new cSenzorTemperature(1000, 0, -40, 1000, 40), 5*60);
 Senzori.Dodaj (new cSenzorTlaka(1004, 0, 900, 500, 1200), 5*60);
 ...
 cTimer Timer(1./60, Prozivatelj.Prozovi);
 while (1);
 return 0;
}
```

# Dijagram prijelaza stanja

- Koristi se da prikaže prostor stanja danog klasa, događaje koji uzrokuju prijelaz iz jednog stanja u drugo i akcije koje proizlaze iz promjene stanja
- budući da svaki klasa nema značajno ponašanje koje je uzrokovano događajima, dijagram prijelaza stanja se definira samo za klase koji takvo ponašanje imaju
- može se načiniti i dijagram prijelaza stanja za cijeli sistem
- dva osnovna elementa dijagrama stanja su :
  - stanje
  - prijelaz između stanja



# Stanja

- stanje objekta predstavlja kumulativni rezultat njegovog ponašanja
- u bilo kojem vremenskom trenutku, stanje objekta sadrži sva njegova (statička) svojstva, i (dinamičke) vrijednosti za svako od tih svojstava
- svako stanje mora imati ime (koje mora biti jedinstveno unutar danog dosega)
- simbol koji prikazuje stanje :



# Prijelazi među stanjima

- događaj je neka pojava koja može uzrokovati promjenu stanja sistema
- ovu promjenu nazivamo *prijelaz među stanjima* i označavamo simbolom :

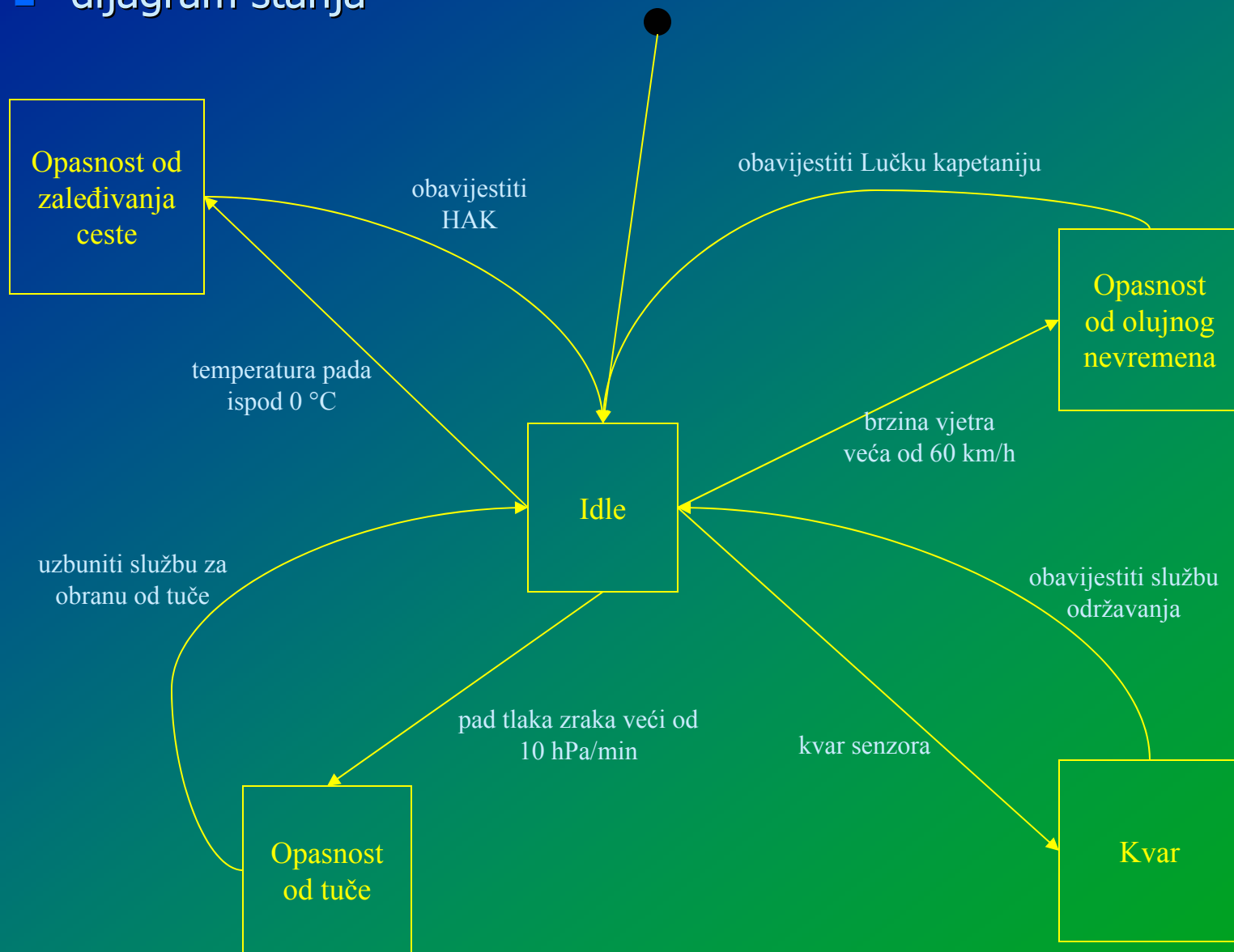


- svaki prijelaz povezuje dva stanja (stanje može imati prijelaz ponovno u isto stanje - *refleksivnost*)
- iz jednog stanja može biti više prijelaza, ali svaki takav prijelaz mora biti jedinstven (ne mogu se istovremeno jednim događajem pokrenuti dva prijelaza iz istog stanja)

- kao primjer nadopunit ćemo našu stanicu za praćenje vremenskih uvjeta sa dodatnim modulima za uzbunjivanje u slučaju pogoršanog vremena
- možemo definirati slijedeće događaje
  - temperatura pada ispod 0°C
  - tlak zraka pada brže od 10 hPa/min
  - brzina vjetra veća od 60 km/h
  - došlo je do kvara nekog od senzora
- događaji će vjerojatno uzrokovati pokretanje neke akcije
- akcija predstavlja postupak koja se praktično obavlja trenutno
- akcija tipično predstavlja poziv neke funkcije, okidanje drugog događaja, ili pokretanje ili zaustavljanje neke aktivnosti
- aktivnost je postupak koji neko vrijeme traje

- događaj može biti simboličko ime, klasa ili ime neke operacije
- akcija može biti opisana uz korištenje slijedeće sintakse :
  - stanica.Uzbuna                      operacija
  - NiskaTemperatura                  okidanje događaja
  - počni                                      započinjanje neke aktivnosti
  - prekini                                  prekidanje neke aktivnosti
- u svakom dijagramu stanja mora postojati jedno početno stanje, iz kojega sistem kreće, označeno ispunjenim crnim kružićem

## ■ dijagram stanja



- prethodni elementi dovoljni su za opisivanje jednostavnih dijagrama stanja
- mogu se definirati i dodatni elementi :
  - specificiranje akcije koja se treba izvesti prilikom ulaska, odn. izlaska iz nekog stanja
    - ulaz     `EvidentirajKvar`
    - izlaz    `EvidentirajObavijestOKvaru`
  - aktivnosti mogu biti povezane sa stanjem, koristeći slijedeću sintaksu
    - čini     `UspostaviKomunikaciju`
  - mogući su i uvjetovani prijelazi među stanjima
    - npr. obavijest o niskoj temperaturi će se poslati samo ukoliko već nije poslana prije nekog definiranog vremena

- ako se dogodi događaj za koji trenutno stanje nema prijelaza znači da je analiza problema bila nepotpuna
- moguće je i ugnijezditi stanja (korisna mogućnost budući da sa porastom složenosti modela dolazi do kombinatornog porasta broja stanja)
- prilikom prijelaza u stanje sa podstanjima, često se želimo vratiti u zadnje posjećeno stanje u tom podstanju
  - ovakvo ponašanje možemo definirati sa slovom H unutar kruga, smještenim bilo gdje unutar stanja

# Objektni dijagrami

- koristi se da se prikaže postojanje objekata i njihovi odnosi u logičkom dizajnu sistema
- svaki objektni dijagram predstavlja međudjelovanja ili strukturalne odnose koji se mogu pojaviti među danim skupom pojavnosti klasa (objekata)
- jedan objektni dijagram predstavlja pogled na objektnu strukturu sistema
- dva bitna elementa objektnih dijagrama su :
  - objekti
  - odnosi među objektima



# Objekti

- simbol objekta



- ime objekta slijedi sintaksu za attribute :
  - A samo ime objekta
  - : C samo klasa objekta
  - A : C ime objekta i klasa objekta
- ako više simbola objekata u istom objektnom dijagramu ima isto ime, oni označavaju isti objekt
- ako simboli objekata u različitim dijagramima imaju isto ime, označavaju različiti objekt

- ukoliko nikada ne specificiramo klasa objekta, tada se klasa objekta smatra anonimnim i nikakva semantička provjera operacija obavljenih nad objektom nije moguća
- ukoliko se samo odredi ime klasa, objekt se smatra anonimnim; svaki simbol bez imena objekta se smatra različitim anonimnim objektom
- za neke objekte se mogu izložiti i neki njihovi pripadni atributi, i vrijedi ista sintaksa

# Odnosi među objektima

- objekti međudjeluju preko veza sa drugim objektima
- simbol za vezu

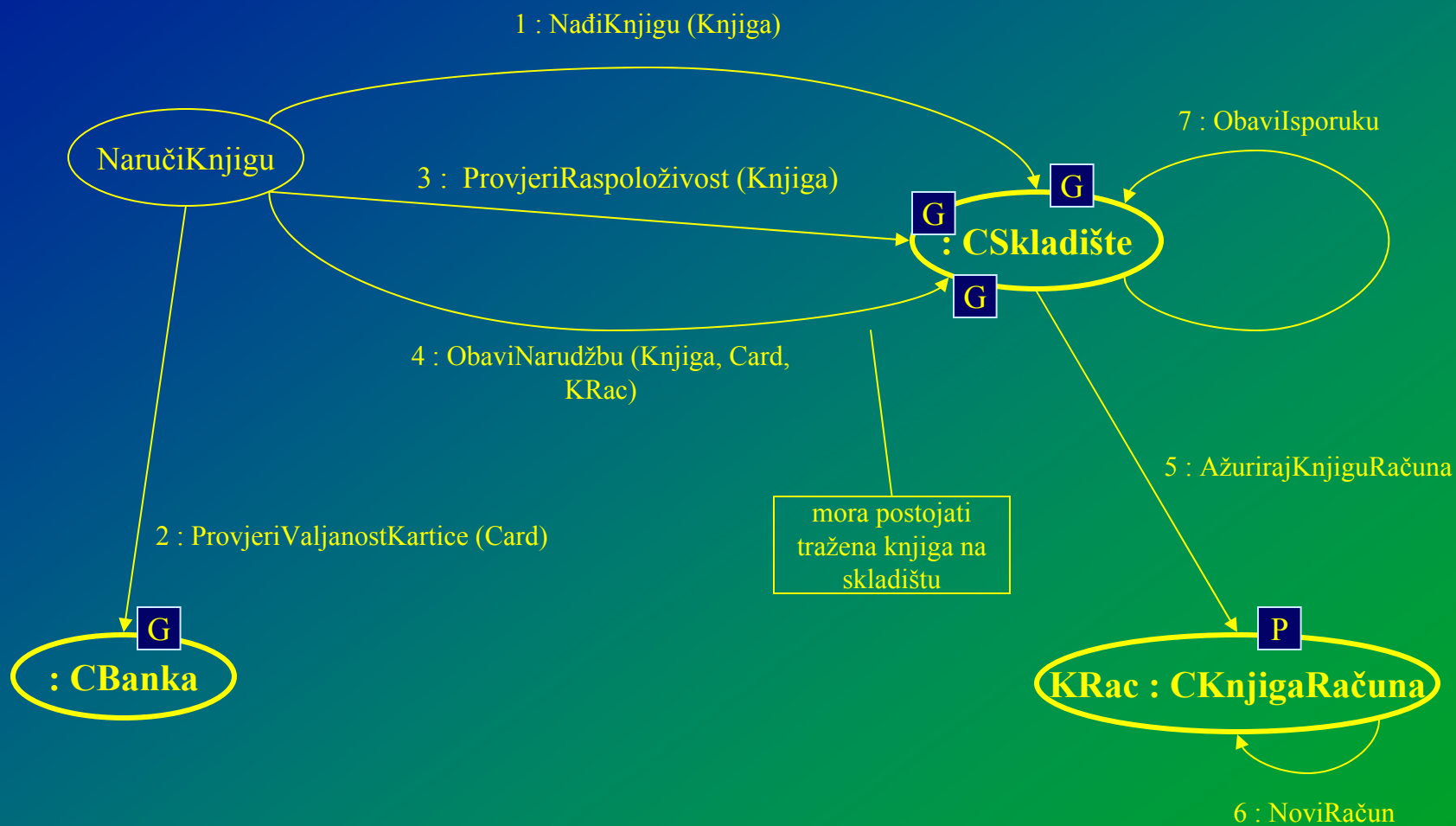


- veza među objektima može postojati samo ako među njihovim klasama postoji asocijacija
- postojanje asocijacije među dva klasa označava put komunikacije među instancama klasa, kojim objekti mogu slati poruke jedni drugima
- smjer slanja poruke se označava strelicom
- najčešći način slanja poruke je pozivanjem neke metode
  - `N()`                      specificiranje samo imena operacije
  - `R N(argumenti)`        specificiranje povratnog objekta, imena i aktualnih argumenata

- prilikom slanja poruka, ako je bitan redoslijed kojim se pozivaju operacije kao odgovori na događaje, može se svakom pozivu pridružiti redni broj
- primjer:



- u nekim složenijim scenarijima korisno je točno pratiti na koji način je jedan objekt vidljiv drugome
- asocijacije u dijagramima klasa označavaju semantičku ovisnost, ali ne govore kako točno instance klasa vide jedna drugu
- na veze koje povezuju dva objekta mogu se dodati simboli koji predstavljaju vidljivost objekta :
  - G poslužitelj je globalan klijentu
  - P poslužitelj je parametar nekoj operaciji klijenta
  - F poslužitelj je dio objekta klijenta
  - L poslužitelj je lokalno deklariran objekt u doseg objektnog dijagrama



# Objektno programiranje u VB

- razvojem vlastitih objekata
  - načini se `cls` modul gdje se definiraju svojstva i postupci
  - deklarira se varijabla definiranog tipa
  - pridjeljuju se svojstva i pokreću postupci
- korištenjem gotovih objekata koji se postavljaju na Ekranski obrazac (*form*): elementi interfejsi, podatkovno povezani objekti, ...
  - objekt se postavlja na obrazac
  - pridjeljuju se svojstva i pokreću postupci
- korištenjem gotovih komponenata iz raznih knjižnica ili drugih programa
  - deklarira se varijabla definiranog tipa
  - pridjeljuju se svojstva i pokreću postupci

## ■ Jednostavni primjer: cOsoba.cls

VB  Osoba0

```
Public Ime As String
Public Prezime As String
Public DatumRodjenja As Date
```

## ■ Korišćenje

```
Dim Pero As New cOsoba
Pero.Ime = "Pero2"
Pero.Prezime = "Perić"
Pero.DatumRodjenja = "01.01.2010"
```

## ■ bolje rješenje

```
Private pIme As String
Private pPrezime As String
Private pDatumRodjenja As Date

Public Property Let Ime (aIme)
 ...
End Sub

Public Property Get Ime () As String
 ...
End Sub
```



## ■ Objekt kao svojstvo objekta

VB  Osoba1

```
Private pIme As String
Private pPrezime As String
Private pDatumRodjenja As Date
Private pOtac As cOsoba
Private pMajka As cOsoba
Public Property Set Otac(inOtac As cOsoba)
 If DateDiff("yyyy", inOtac.DatumRodjenja, pDatumRodjenja) < 10 Then
 Err.Raise vbObjectError + 4, "cOsoba", inOtac.ImeIPrezime & _
 " ne može biti otac od " & ImeIPrezime
 End If
 Set pOtac = inOtac
End Property
Public Property Get Otac() As cOsoba
 Set Otac = pOtac
End Property
...
Dim Ivo As New cOsoba
Ivo.Ime = ...
Set Pero.Otac = Ivo
```

## ■ Stvaranje objekta

### Deklaracijom varijable

```
Dim O As cOsoba
```

deklarira se samo referenca na objekt, koji još ne postoji.

```
Debug.Print O Is Nothing
```

⇒ True

Referenci je potrebno pridružiti postojeći ili novi objekt:

```
Set O = Pero
```

```
Set O = New cOsoba
```

Deklaracija i pridruživanje novog objekta:

```
Dim O As New cOsoba
```

Stvaranjem novog objekta obavlja se procedura

```
Private Sub Class_Initialize()
```

```
...
```

```
End Sub
```

## ■ Uništavanje objekta

- eksplicitno

```
Set O = Nothing
```

- implicitno: po izlasku iz potprograma u kojem je deklariran, ako ne postoji važeća referenca

- Primjer:

```
Private O As cOsoba
Private Sub P ()
 Dim A As New cOsoba
 Set O = A
End Sub
```

## ■ Uništavanjem objekta obavlja se procedura

```
Private Sub Class_Terminate()
 ...
End Sub
```

# Zbirke objekata (*collections*)

## ■ Deklaracija

VB  Collection

```
Dim Osobe As New Collection
```

## ■ Dodavanje objekta u zbirku

Sintaksa: *object.Add item, [,key] [, before] [, after]*

```
Dim A As New cOsoba, B As New cOsoba, C As New cOsoba
```

```
A.Ime = "Ivo" ...
```

```
Osobe.Add A, Trim(A.Mbr)
```

```
Osobe.Add B, Trim(B.Mbr)
```

```
Osobe.Add C, Trim(C.Mbr)
```

## ■ Broj objekata u zbirci

```
Debug.Print Osobe.Count
```

## ■ Dohvat objekta iz zbirke

```
Set O = Osobe (1)
```

```
Debug.Print Osobe(3).Ime
```

```
Debug.Print Osobe("202").Ime
```

## ■ Uklanjanje objekta iz zbirke

```
Osobe.Remove 3
```

```
Osobe.Remove "202"
```

## ■ Iteracija po zbirci

```
Dim I As Integer
For I = 1 To Osobe.Count
 Debug.Print Osobe(I).Ime
Next I
```

ili

```
Dim O As cOsoba
For Each O In Osobe
 Debug.Print O.Ime
Next O
```

## ■ Brisanje svih elemenata

```
For I = Osobe.Count To 1 Step -1
 Osobe.Remove I
Next I
```

ili

```
For Each O In Osobe
 Osobe.Remove 1
Next O
```

## ■ Brisanje cijele zbirke

```
Set Osobe = Nothing
```

## ■ Vlastite zbirke

### ● Primjer: cOsobe.cls

```
Option Explicit
```

```
Private C As New Collection
```

```
Public Function Add(V As cOsoba, Key As Variant) As cOsoba
```

```
 C.Add V, Trim(Key)
```

```
 Set Add = V
```

```
End Function
```

```
Public Sub Remove(Key As Variant)
```

```
 C.Remove Key
```

```
End Sub
```

```
Public Function NewEnum() As IUnknown
```

```
 Set NewEnum = C.[_NewEnum]
```

```
End Function
```

```
Public Function Item(ByVal Index As Variant) As cOsoba
```

```
 Set Item = C.Item(Index)
```

```
End Function
```

```
Public Function Count() As Long
```

```
 Count = C.Count
```

```
End Function
```

## ■ Zbirke raznorodnih objekata

```
Dim A As New cAuto
Dim B As New cOsoba
Dim X As Object
Dim C As New Collection
C.Add A
C.Add B
For Each X In C
 If TypeOf X Is cAuto Then
 Debug.Print X.Proizvodjac
 Else
 Debug.Print X.Ime
 End If
Next X
```

# Nasljeđivanje i višeobličje u VB

Most object-oriented programming tools provide polymorphism through inheritance. This is a powerful mechanism for small-scale development tasks, but has generally proven to be problematic for large-scale systems.

- In part, these difficulties arise as a result of necessary changes to classes deep in the inheritance tree. Recompile is required in order to take advantage of such changes, and failure to recompile may lead to unpleasant surprises when the time finally arrives for a new version.
- More seriously, an over-emphasis on inheritance-driven polymorphism typically results in a massive shift of resources from development tasks to up-front design tasks, doing nothing to address development backlogs or to shorten the time before the end user can discover — through hands-on experience — whether the system actually fulfills the intended purpose.
- As a consequence, tools for rapid prototyping and Rapid Application Development (RAD) have gained wider acceptance than OOP tools.



## ■ Višeobličje preko klasa Object

```
Sub Pomakni (O As Object, X As Integer, Y As Integer)
 O.Move X, Y
 If TypeOf O Is TextBox Then
 ...
 End If
End Sub
```

- Kasno povezivanje (*late binding*)

## ■ Višeobličje preko interfejsi

VB  Interface0

- cLik.cls

```
Public Property Set Podloga(inPodloga As Form)
End Property

Public Sub Parametri(inA As Integer, inX As Integer, inY As Integer)
End Sub

Public Property Get Povrsina() As Integer
End Property

Public Sub Nacrtaaj()
End Sub
```

- cKrug.cls

Implements cLik

```
Public Property Get cLik_Povrsina() As Integer
```

```
 cLik_Povrsina = R * R * 3.141592653
```

```
End Property
```

...

- Korištenje:

```
Dim Krug As cKrug
```

```
Dim Lik As cLik
```

```
Set Lik = Krug
```

```
Lik.Parametri 100, 100, 100
```

```
Print Lik.Povrsina
```

- Rano povezivanje (*early binding*)

- Alternativno: *subclassing*

VB  Interface1

- cKrug.cls

```
Private Lik As New cLik
```

```
Public Property Get Povrsina() As Integer
```

```
 Povrsina = Lik.A * Lik.A * 3.141592653
```

```
End Property
```

# Objekti s događajima (*events*)

■ cPosao.cls

VB  Events

```
Event Pocetak()
Event PostoGotovo(Posto As Integer)
Event Kraj()
Public Sub Radi()
 Dim I As Long
 RaiseEvent Pocetak
 KrajPetlje = 5000000
 For I = 1 To KrajPetlje
 If I Mod (KrajPetlje/100) = 0 Then
 RaiseEvent PostoGotovo(I / KrajPetlje * 100)
 End If
 Next I
 RaiseEvent Kraj
End Sub
```

## ■ Korištenje

```
Private WithEvents Posao As cPosao
Private Sub Command1_Click()
 Set Posao = New cPosao
 Posao.Radi
End Sub
Private Sub Posao_Pocetak()
 PB.Visible = True
End Sub
Private Sub Posao_PostoGotovo(Posto As Integer)
 PB.Value = Posto
End Sub
Private Sub Posao_Kraj()
 PB.Visible = False
End Sub
```

## ● Primjer: VB Timer

# Objekti izvori podataka (*data sources*)

## ■ cIzvorPodataka.cls

### ● Svojstvo objekta:

DataSourceBehavior = 1-VBDataSource

```
Private rs As ADODB.Recordset
```

```
Private Sub Class_Initialize()
```

```
 Set rs = New ADODB.Recordset
```

```
 With rs
```

```
 .Fields.Append NazivAtributa, adInteger
```

```
 ...
```

```
 .CursorType = adOpenStatic
```

```
 .LockType = adLockOptimistic
```

```
 .Open
```

```
 End With
```

```
End Sub
```

```
Private Sub Class_GetDataMember(DataMember As String, Data As Object)
```

```
 Set Data = rs
```

```
End Sub
```

## ■ Korišćenje na obrascu:

```
Private Izvor As New cIzvorPodataka
Private objBindingCollection As New BindingCollection

Private Sub Form_Load()
 Set objBindingCollection.DataSource = Izvor
 objBindingCollection.Add ImeKontrole, "Text", NazivAtributa
End Sub
```

# Objekti korisnici podataka (*data consumers*)

## ■ cKorisnikPodataka.cls

```
DataBindingBehavior = 1-VBSimpleBound (ili 2-VBComplexBound)
```

```
Private Property Let ImeSvojstva (Vrijednost As TipPodataka)
```

```
...
```

```
End Sub
```

## ■ Korišćenje

```
Private Korisnik As New cKorisnikPodataka
```

```
Private objBindingCollection As New BindingCollection
```

```
...
```

```
objBindingCollection.DataSource = IzvorPodataka
```

```
objBindingCollection.Add Korisnik, ImeSvojstva, NazivAtributa
```

```
...
```

## ■ Primjer: VB DataAware

# Dinamičko stvaranje objekata interfejsi

- VB 5.0: mora postojati objekt koji je element polja, npr. sa svojstvom

```
Index = 0
```

```
...
```

```
Load ImeObjekta(1)
```

```
ImeObjekta(1).Visible = True
```

- VB 6.0

```
...
```

```
Dim C As Object
```

```
Set C = Obrazac.Controls.Add(TipObjekta, ImeObjekta)
```

```
C.Left = ...
```

```
C.Top = ...
```

```
C.Height = ...
```

```
C.Width = ...
```

```
C.Visible = True
```

```
C.Svojstvo = ...
```

- **Primjer:** VB  Data



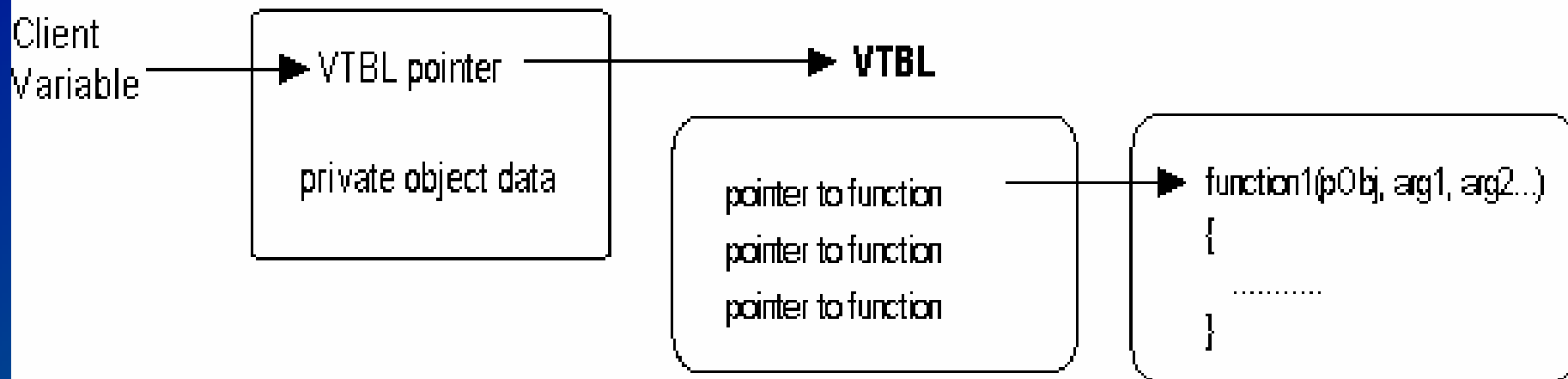
# Component Object Model (COM)

- C++ prikladan za pisanje programa koji se prevode odjednom, tj. za ponovno iskorištenje izvornog kôda. Ako se promijeni definicija klasa, program treba ponovo prevesti.
  - problem novih inačica velikih programa: dugo prevođenje, potrebno je korisniku dostaviti cijeli program.
- djelomično rješenje: knjižnica (DLL).
  - problem kod povezivanja programa pisanih u različitim jezicima – različite konvencije.
  - problem promjene definicije klasa: dodavanje nove virtualne funkcije nije moguće niti na kraj *vtable*, jer treba promijeniti *vtable* i svih izvedenih klasa, kao i klijenta.
- Java
  - korištenje komponenata moguće samo iz Java programa (osim uz *bridge*)
  - različito korištenje lokalnih i udaljenih komponenata
  - bitno sporija za od C++

# Component Object Model (COM)

- standard za razvoj softverskih komponenata koje međusobno komuniciraju
- nezavisan od hardverske platforme i programskog jezika
  - jezik mora podržavati strukture i pokazivače, te poziv funkcija preko pokazivača
- objekt podržava jedan ili više skupova funkcija: jedno ili više interfejsi (*interface*)
- interfejs može i ne mora sadržavati sve ugrađene funkcije
- pristup postupcima objekta preko pokazivača na *vtable* (*virtual function table*) koja sadrži pokazivače na funkcije
- za svaki hardver i operacijski sistem definira se format *vtable* i način pozivanja funkcija

## Component Object

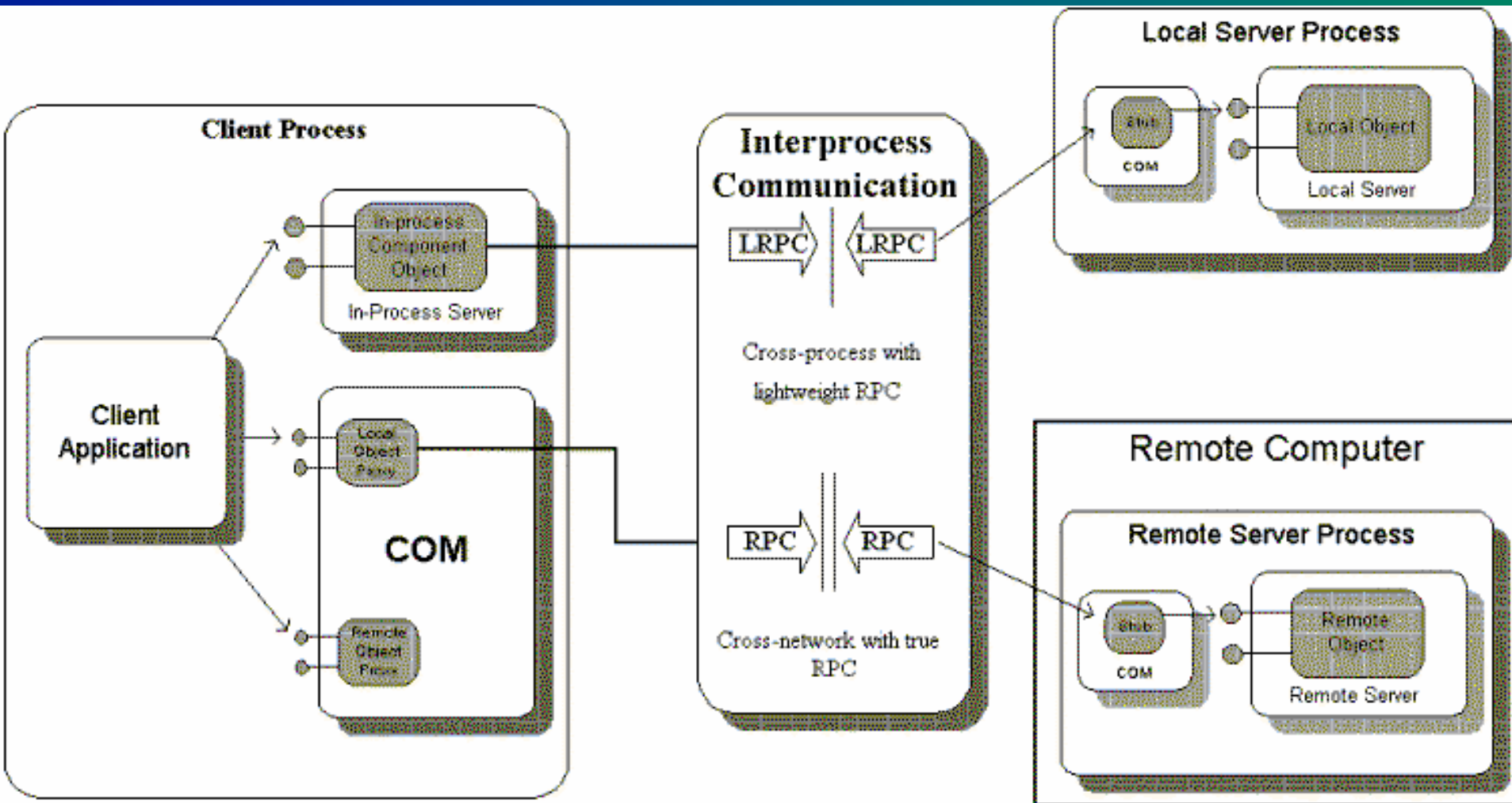


- svako interfejs ima vlastiti identifikator: 128-bitni integer GUID (globally unique ID) prema definiciji Open Software Foundation's Distributed Computing Environment
- IID identifikator interfejsi
- CLSID identifikator klasa
- modificirana verzija interfejsi dobiva novi GUID čime je omogućena kompatibilnost s postojećim klijentima

- svaki COM objekt ima IUnknown interfejs

```
interface IUnknown {
 virtual HRESULT QueryInterface(IID& iid, void** ppvObj)
 = 0;
 virtual ULONG AddRef() = 0;
 virtual ULONG Release() = 0;
}
```

- COM omogućuje transparentni poziv komponente iz adresnog prostora procesa (*in-process call*), izvan adresnog prostora procesa (*out-of-process call*) i na drugom stroju (*remote call*)
- poziv izvan adresnog prostora i na drugom stroju omogućuju *proxy* i *stub*



# ActiveX komponente u VB

- ActiveX komponenta: izvršni program koji izlaže objekte za korištenje u drugim programima
- VB program može biti
  - Standard EXE
  - ActiveX EXE Primjer: VB  ActiveX
  - ActiveX DLL
  - ActiveX Control Primjer: VB  UserControl
- klase u ActiveX EXE mogu biti
  - *private* - ostali programi ne mogu koristiti objekt
  - *public not creatable* - ostali programi mogu koristiti ali ne i stvoriti objekt
  - *single use* - svaki program pokreće novu kopiju komponente
  - *global single use* - kao *single use*, ali se postupci i svojstva mogu koristiti kao globalne funkcije
  - *multi use* - jedna kopija komponente podržava više objekata
  - *global multi use* - kao *multiuse*, ali se postupci i svojstva mogu koristiti kao globalne funkcije

# Integracija s okružjem MS Office

- MS Word
  - Primjer: VB  Word, Word1
- MS Excel
  - Primjer: VB  Excel
- MS Powerpoint
  - Primjer: VB  PPT
- MS Outlook
  - Primjer: VB  Outlook

# Neki detalji programiranja u C++

- Koristit će se primjer klasa Datum

```
class Datum {
 int dan, mj, god;
public:
 Datum (int dd=0, int mm=0, int gg=0);
 void Init(int ddan, int mmj, int ggod);

 void DodajGod (int n);
 void DodajMj (int n);
 void DodajDan (int n);
};

void Datum::DodajGod (int n) {
 god += n;
}
```



# Statički članovi klasa

- Ukoliko se ne definiraju parametri konstruktora, potrebno je postaviti vrijednost datuma na neku pretpostavljenu vrijednost
- može se, npr. definirati globalnu varijablu `danas` koja sadrži vrijednost današnjeg datuma

```
Datum::Datum(int dd, int mm, int gg) {
 {
 // provjera valjanosti datuma
 ...
 dan = dd ? dd : danas.dan;
 mj = mm ? mm : danas.mj;
 god = gg ? gg : danas.gg;
 }
}
```

- problem je što sada program ovisi o globalnoj varijabli `danas`
- rješenje je definiranje `static` člana klasa

- `static` član klase je varijabla koja je dio klase, ali nije dio objekta date klase
- sve instance date klase dijele zajedničku (jednu) instancu `static` varijable
- slično, funkcija koja treba pristupati statičkim članovima klase, ali se ne može pozivati za neki određen objekat naziva se `static` članska funkcija

```
class Datum {
 int dan, mj, god;
 static Datum default_datum; // statička varijabla
public:
 Datum (int dd=0, int mm=0, int gg=0);
 static void PostaviDefault(int, int, int); // statička funkcija
 //...
}
```

```
Datum::Datum(int dd, int mm, int gg) {
 // provjera valjanosti datuma
 ...
 dan = dd ? dd : default_datum.dan;
 mj = mm ? mm : default_datum.mj;
 god = gg ? gg : default_datum.god;
}
```

```

static void Datum::PostaviDefault(int dd, int mm, int gg) {
 default_datum.dan = dd;
 default_datum.mj = mm;
 default_datum.god = gg;
}

// poziv funkcije
void main () {
 PostaviDefault (1,1,1999);
 ...
}

```

## ■ Za public static varijable, npr.

```

class Datum {
 int dan, mj, god;
public:
 static Datum default_datum; // statička varijabla
 ...
}

void main () {
 default_datum = Datum (24, 12, 1998);
 ...
}

```

# Const članske funkcije

- klasi `Datum` treba dodati i članske funkcije za dohvat trenutne vrijednosti datuma

```
class Datum {
 int dan, mj, god;
public:
 int Dan() const { return dan; }
 int Mjesec() const { return mj; }
 int Godina() const { return god; }
};
```

- `const` deklarator nakon liste argumenata kazuje da funkcija ne mijenja stanje objekta klase `Datum` za koji je pozvana
- ukoliko se funkcija definira izvan klase, potrebno je eksplicitno navesti `const` kvalifikator

```
int Datum::Godina () const {
 return god;
}
```

- `const` članska funkcija se može pozvati i za `const` i za `ne-const` objekte, dok se `ne-const` funkcija može pozvati samo za `ne-const` objekte

```
void func(Datum &d, const Datum &cd)
{
 cout << d.Godina() << ' ' << cd.Godina()
 d.DodajGod(1); // OK
 cd.DodajGod(1); // Pogreška : ne može se mijenjati
 // vrijednost const cd
}
```

## this pokazivač

- funkcije `DodajDan()`, `DodajMjesec()` i `DodajGodinu()` nisu vraćale vrijednosti
- budući da su dane funkcije povezane, bilo bi dobro da vraćaju referencu na promijenjeni objekt tako da se operacije mogu ulančati
- npr:

```
void f (Datum &d) {
 d.DodajGodinu(1).DodajMjesec(1).DodajDan(1);
}
```

- treba promijeniti definiciju klase `Datum`

```
class Datum {
 Datum & DodajGodinu (int n);
 Datum & DodajMjesec (int n);
 Datum & DodajDan (int n);
};
```

- svaka ne-statička članska funkcija zna za koji je objekt pozvana i može se eksplicitno pozvati na njega

```
Datum & Datum::DodajGodinu (int n) {
 if(dan == 29 && mj == 2 && !PrestupnaGod(god + n) {
 dan = 1;
 mj = 3;
 }
 god += n;
 return *this;
}
```

- u ne-const članskoj funkciji klasa X, tip od this je X \* const (zato da se eksplicitno naznači da nije dozvoljeno mijenjati this)
- u gornjoj funkciji se ne može napisati

```
this = new Datum(); // pogreška
```

- ali može

```
if(this->dan == 29 && this->mj==2 && !PrestupnaGod(this->god + n)
```

- za `const` člansku funkciju klase `X`, tip od `this` je `const X *const` (da se spriječi promjena samog objekta na koji pokazuje `this`)

```
int Datum::Godina () const {
 this->god = 1990; // pogreška
 return god;
}
```



# inline - funkcije

- kod `inline` funkcija umeće se njihov kôd direktno na mjesto odakle je funkcija pozvana (za razliku od običnih funkcija čiji kod se nalazi na jednom mjestu u programu a funkcija se onda poziva prijenosom parametara preko stoga)
- za male, često korištene funkcije
- dva načina definiranja `inline` funkcija:

- ukoliko se funkcija definira unutar klase

```
class Datum {
 //...
 int Godina() { return god; }
}
```

- ukoliko se prilikom definiranja funkcije eksplicitno navede `inline` kvalifikator

```
inline int Datum::Godina() {
 return god;
}
```

# Konstruktori i destruktori

- konstruktor inicijalizira objekt
  - obično je potrebno alocirati sistemske resurse
- destruktor oslobađa alocirane resurse
- destruktori se pozivaju :
  - implicitno kada varijabla izađe van dosega
  - kada se briše objekt na *heap-u*
  - eksplicitnim pozivom (rijetko)

```
class Table {
 string * p;
 int sz;
public:
 Table(int a = 5) { p = new string[a]; sz = a; }
 ~Table() { delete [] p; }

 //..
};
```

- ukoliko korisnik ne definira konstruktor, koristi se podrazumijevani konstruktor (koji se poziva bez ikakvih argumenata)
  - podrazumijevani konstruktor implicitno poziva konstruktore za članove klase koji su sami klase
- primjer

```
class Table { };
```

```
struct Tables {
 int i;
 int vi[10];
 Table t1;
 Table vt[10];
};
```

```
Tables tt;
```

- pozvati će se konstruktor `Table()` za `tt.t1` i za svaki element `tt.vt`
- ali, `tt.i` i elementi od `tt.vi` se neće inicijalizirati jer nisu klase

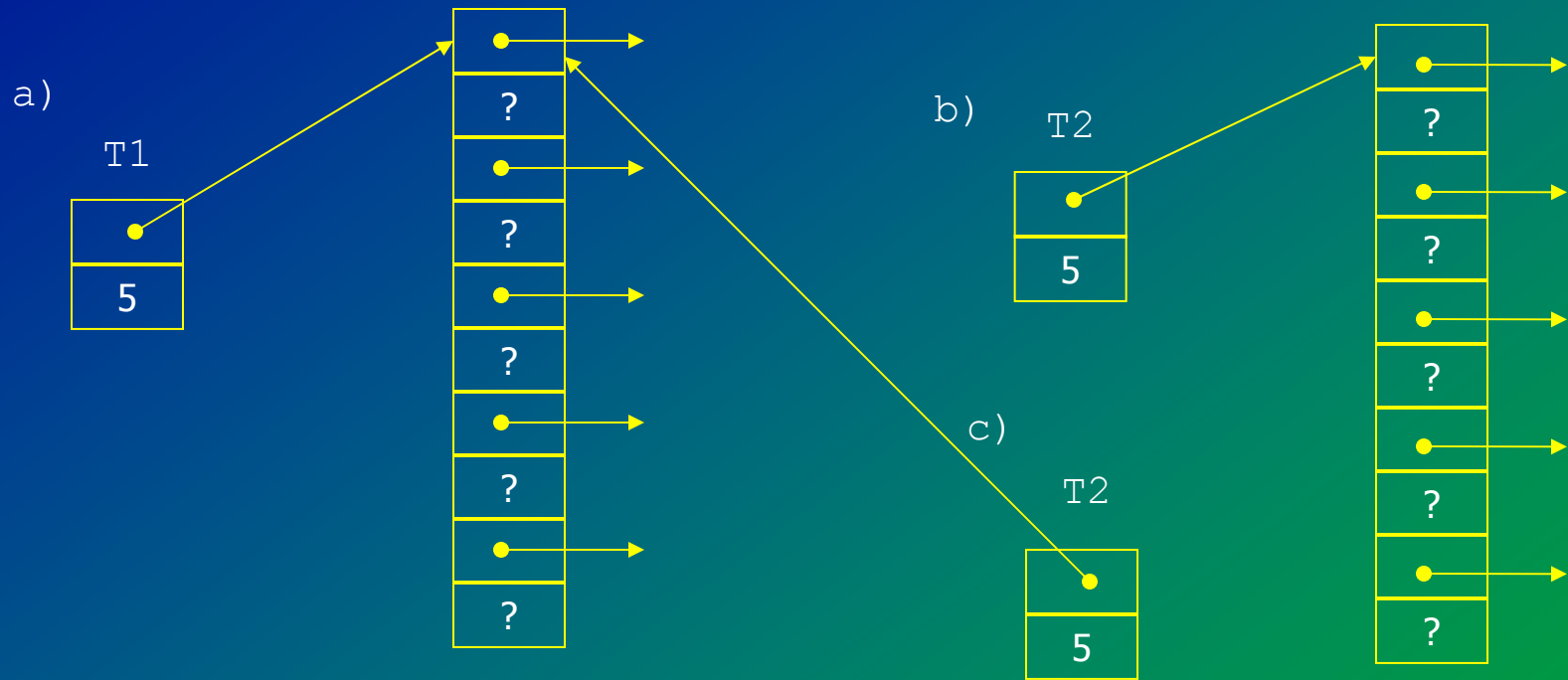
# Kopiranje objekata

- ukoliko `t1` i `t2` predstavljaju dva objekta klase `Table`, `t2=t1` znači kopiranje vrijednosti član po član
  - problem ukoliko imamo članske varijable koje su pokazivači

```
void h() {
 Table t1; // a) inicijalizacija
 Table t2 = t1; // b) inicijalizacija c) kopiranje
 Table t3;

 t3 = t2; // pridruživanje kopiranjem
}
```

- na kraju funkcije `t1`, `t2` i `t3` pokazuju na polje alocirano u konstruktoru pozvanom za objekt `t1`
  - prilikom poziva destruktora po izlazu iz funkcije, destruktor za to polje će biti pozvan tri puta
- nijedan objekt više ne sadrži pokazivač na polje alocirano za objekte `t2` i `t3`



- mogu se definiraty *copy* konstruktor i operator pridruživanja

```
class Table {
 //...
 Table(const Table &);
 Table & operator=(const Table&);
};

Table::Table(const Table & t) {
 p = new string[sz = t.sz];
 for (int i=0; i<sz; i++)
 p[i] = t.p[i];
}

Table & Table::operator=(const Table & t) {
 if (this != t) { // treba paziti na t=t
 delete [] p;
 p = new string[sz = t.sz];
 for (int i=0; i<sz; i++)
 p[i] = t.p[i];
 }
}
```

- treba razlikovati *copy* konstruktor i pridruživanje kopiranjem

# Objekti kao članovi klasa

- primjer klasa koji sadrži podatke o maloj organizaciji

```
class Klub {
 string Ime;
 Table Clanovi;
 Table Sluzbenici;
 Datum DatOsnivanja;

 Klub(const string &, Datum d);
};
```

- pretpostavljeni konstruktori za članove automatski se redom kojim su navedeni pozivaju iz konstruktora za sam klasa
- redoslijed i izbor konstruktora kod nekih se kompajlera (ne VC++) može eksplicitno promijeniti

```
Klub::Klub(const string &ImeKluba, Datum d)
 : Ime(ImeKluba), DatOsnivanja(d), Clanovi(), Sluzbenici() {
 //...
}
```

- ukoliko konstruktor za člansku varijablu ne zahtijeva argumente, može se izostaviti iz liste
- prilikom destrukcije objekta, najprije se izvršava destruktor za sam klasa, a onda za članske varijable, i to obrnutim redoslijedom od onog kojim su deklarirani

- na drugi način se gornji konstruktor mogao definirati kao :

```
Klub::Klub(const string &n, Datum d) {
 Ime = n;
 DatOsnivanja = d;
}
```

- međutim, tada je nužno postojanje operatora pridruživanja (pridruživanje kopiranjem)
  - manja efikasanost jer se najprije Ime i DatOsnivanja inicijaliziraju sa praznim (*default*) konstruktorima, a tek im se onda pridjeljuje vrijednosti



# Polja

- polje objekata nekog tipa možemo definirati SAMO ukoliko se objekti danog tipa mogu inicijalizirati sa default konstruktorom
- primjer :

```
Table tbl[10];
```

- ne postoji način da se odrede eksplicitni argumenti za konstruktor prilikom deklaracije polja
- prilikom destrukcije polja, poziva se destruktorka za svaki član
- budući da C++, kao i C, ne razlikuje pokazivače na individualan element i pokazivače na polje, prilikom poziva destruktora je potrebno eksplicitno reći da li se poziva za polje

```
Table *t1 = new Table;
Table *t2 = new Table[10];
delete t1;
delete[] t2;
delete t2; // pogreška
```

# Privremeni objekti

- najčešće su rezultat izračunavanja izraza i rijetko zahtijevaju posebnu pažnju (osim kad je bitna efikasnost)
- privremeni objekt sadrži rezultat izračunavanja nekog podizraza i uništava se nakon izračunavanja cijelog izraza
- primjer:

```
void f(string &s1, string &s2, string &s3) {
 const char *cs, ct;
 cs = (s1+s2).c_str();
 cout << cs; // vjerojatno će raditi
 ct = (s2+s3).c_str();
 cout << cs << ct;

 if(strlen(cs=(s2+s3).c_str()) < 8 && cs[0] == 'a') { // OK
 ...
 }
}
```

- problem je što se tipovi visokog nivoa apstrakcije koriste na način koji odgovara niskom nivou

# Smještanje objekata

- pomoću operatora `new` se alocira prostor za objekt na *heap*-u

```
Table *t1 = new Table(10);
```

- međutim, uz pomoć dodatne sintakse, moguće je i točno odrediti mjesto na kojem će se u memoriji nalaziti objekt

```
void *buf = 0xFFFFF;
Table *t1 = new (buf) Table(10);
```

- nakon ovoga će se objekt `t1` nalaziti upravo na specificiranoj adresi `0xFFFFF`

# Konstruktori i konverzije

- pomoću konstruktora možemo definirati implicitnu konverziju parametara u traženi tip podatka
- primjer klasa `complex`

```
class complex {
 // ...
 complex operator+(complex, complex);
 complex operator+(complex, double);
 complex operator+(double, complex);
};
```

- budući da želimo da tip `complex` ravnopravno sudjeluje u izrazima sa tipom `double`, potrebno je definirati sve tri gornje navedene funkcije
- drugo rješenje je da se oslonimo da konverziju pomoću definiranog konstruktora, koji iz `double` kreira `complex`

```
complex(double re) { real = re; imag = 0.0; }
```

- sada se može definirati samo jednu funkciju usporedbe

```
bool operator==(complex, complex);

void f(complex x, complex y)
{
 if (x == y) ...; // operator==(x, y)
 if (x == 3.) ...; // operator==(x, complex(3))
 if (3. == y) ...; // operator==(complex(3), y)
}
```

- ponekad je bolje koristiti posebne funkcije
- u slučaju da postoji više mogućnosti za obavljanje konverzije, prevodilac mora odabrati pravu na osnovu argumenata i raspoloživih (standardnih i korisnički definiranih) konverzija

# Operatori konverzije

- korištenje konstruktora da se definira konverzija među tipovima ima sljedeće nedostatke

- ne može se specificirati implicitna konverzija iz korisnički definiranog tipa u osnovni tip (jer osnovni tipovi nisu klase)

```
int i, Datum d; i = d;
```

- ne može se definirati konverzija iz novog klase u prethodno definirani klase

```
class A {
 A operator+(A,A);
 ...
};
class B {
 B(A); // ali to je konverzija iz A u B.
}
```

- Kako načiniti konverziju iz B u A da bi, npr. mogli pozvati A+B
- Trebalo bi dodati u klase A konstruktor A(B), odnosno treba modificirati već postojeći klase

- navedeni problemi se mogu riješiti definiranjem *operatora konverzije*
- definira se kao `X::operator T()` gdje je `T` ime tipa u koji se vrši konverzija (dakle iz `X` u `T`)

- primjer : klasa `MaliInt` za 8-bitne cijele brojeve

```
class MaliInt {
 char Vrijednost;
 void Pridruzi(int i) { Vrijednost = i; // uz kontrolu }
public:
 MaliInt(int i) { Pridruzi(i); }
 MaliInt &operator=(int i) { Pridruzi(i); return *this; }

 operator int() const { return Vrijednost; }
};
```

- definirali smo operator konverzije `MaliInt::operator int()` kako bi mogli obavljati uobičajene operacije sa cijelim brojevima nad objektima klasa `MaliInt`
- kad god se u izrazu pojavi `MaliInt` gdje je potreban `int`, obavlja se konverzija

# Nejasnoće kod konverzija

- ukoliko se pretjera sa definiranjem operatora konverzija, može doći do nejasnoća

```
int operator+(MaliInt, MaliInt);

void f(MaliInt t, int i){
 cout << t + i; // pogreška, nejasno: operator+(t, MaliInt(i))
 ili int(t)+i ???
}
```

- u nekim slučajevima, željeni tip se može konstruirati uzastopnom primjenom konstruktora ili operatora konverzije
- ukoliko postoji mogućnost da se željeni tip konstruira na više od jednog načina, javlja se pogreška



```

class X {
 X(int);
 X(char *);
};
class Y {
 Y(int);
};
class Z {
 Z(X);
};

X f(X);
Y f(Y);
Z g(Z);

void func() {
 f(1); // pogreška, nejasno f(X(1)) ili f(Y(1))
 f(X(1)); // ok
 f(Y(1)); // ok

 g("Vještica"); // pogreška: potrebne dvije korisnički
 // definirane konverzije:
 // g(Z(X("Vještica"))) se ne pokušava
 g(X("Ivica")); // ok: g(Z(X("Ivica")))
 g(Z("Marica")); // ok: g(Z(X("Marica")))
}

```

- korisnički definirane konverzije se koriste samo kada su potrebne da se riješi način poziva
- ukoliko se način poziva može riješiti samo pomoću standardnih konverzija, onda se korisnički definirane uopće ne razmatraju

```
class XX {
 XX(int);
};
```

```
void h(double);
void h(XX);
```

```
void k2() {
 h(1); // h(double(1)) ili h(XX(1)) ???
}
```

# Primjer pismenog ispita iz OOP-a

1. Koje su suprotne tendencije prisutne između principa enkapsulacije i hijerarhije (nasljeđivanja) ? (15 bodova)

Osnovna zamisao enkapsulacije je skrivanje svih svojstava objekta unutar definicije klase, dajući prema van samo interfejs (*interface*) pomoću kojeg se može dohvatiti (saznati) trenutno stanje objekta.

Međutim, nasljeđivanje zahtijeva otvaranje članova klase prema van kako bi izvedeni klase mogli modificirati svojstva koja su naslijedili od klase-roditelja čime se razbija enkapsulacija klase-roditelja.

2. a) Objasnite razliku između statičkog i dinamičkog povezivanja ? (10 bodova)

Kod statičkog povezivanja su tipovi svih varijabli i izraza poznati prilikom prevođenja (tipovi su statički, fiksirani i ne mijenjaju se).

Kod dinamičkog povezivanja tipovi nisu poznati sve do trenutka izvođenja programa.

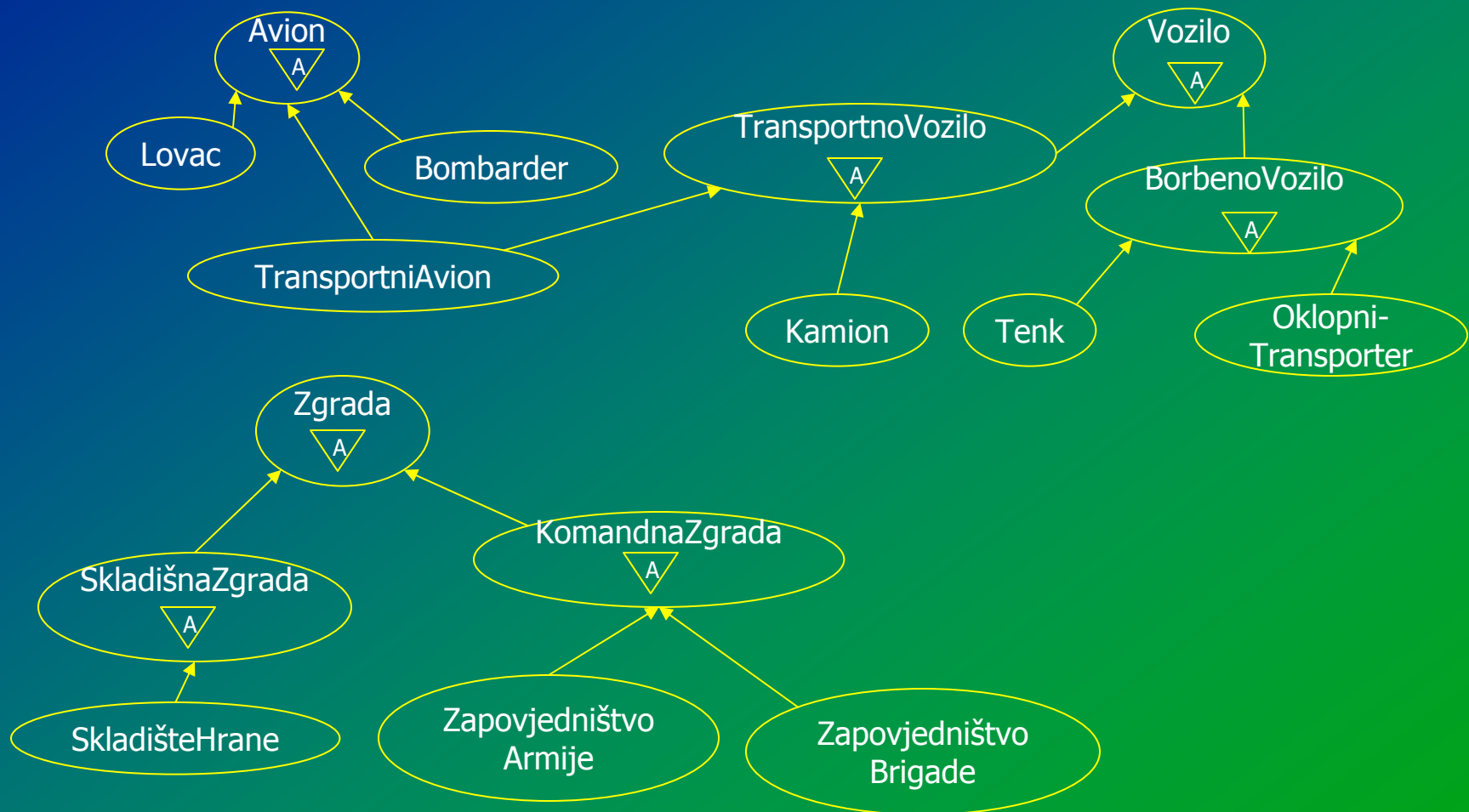
b) Koje se funkcije u sljedećem primjeru statički povezuju, a koje dinamički ?  
(10 bodova)

```
class Oruzje {
 virtual void Napuni();
 virtual void SpremnoZaPaljbu();
};
class Top : public Oruzje {
 virtual vodi Napuni ();
};
class Revolver : public Oruzje {
 virtual vodi Napuni();
};
void Pripremi (Oruzje &a) {
 if(a.SpremnoZaPaljbu() != true)
 a.Napuni();
}
```

Funkcija `SpremnoZaPaljbu()` se povezuje statički zato jer, bez obzira kojeg tipa bio objekt `a` u funkciji `Pripremi()`, funkcija `SpremnoZaPaljbu()` je definirana samo kao član osnovnog klase `Oruzje` i nije član niti jednog izvedenog klase, te se točno zna za koji klasa se poziva ta funkcija.

S druge strane, funkcija `Napuni()` se povezuje dinamički zato što objekt `a` u funkciji `Pripremi()` može biti i tipa `Oruzje`, ali može biti i jednog od izvedenih tipova `Top` ili `Revolver`.

3. Modelirajte sistem za simulaciju ratnih operacija. Definirajte objekte (*Avion*, *BorbenoVozilo*, *Tenk*, *TransportniAvion*, *Vojnik*, *Zgrada*, *Skladište*, ...) koje smatrate osnovnim i postavite njihovu hijerarhiju (dijagram klasa). Uočite i kao takve definirajte sve moguće apstraktne klase. (*Napomena* : pisanje specifikacije i kôda deklaracija klasa nije potrebno). (25 bodova)



4. Za **parametrizirani** (*template*) klasa `Matrix`, deklarirajte i implementirajte konstruktor, *copy* konstruktor i operator pridruživanja. (*Napomena*: pazite na to da se radi o parametriziranom klasi). (25 bodova)

```
template<class T> class Matrix {
public:
 Matrix(int RowNum, int ColNum);
 Matrix(const Matrix &c);
 Matrix & operator=(const Matrix &b);
private:
 int Rows, Cols;
 T **El;
};

template<class T> Matrix<T>::Matrix(int RowNum, int ColNum) {
 El = new T*[RowNum];
 for(int i=0; i<RowNum; i++)
 El[i] = new T[ColNum];
 Rows = RowNum;
 Cols = ColNum;
}
```

```

template<class T> Matrix<T>::Matrix(const Matrix &c) {
 El = new T*[c.Rows];
 for(int i=0; i<c.Rows; i++)
 El[i] = new T[c.Cols];
 Rows = c.Rows; Cols = c.Cols;
 for(i=0; i<Rows; i++)
 for(int j=0; j<Cols; j++)
 El[i][j] = c.El[i][j];
}

template<class T> Matrix<T> Matrix<T>::operator=(const Matrix &b) {
 int i, j;
 if(this != &b) {
 if(Rows != b.Rows || Cols != b.Cols) {
 for(i=0; i<Rows; i++)
 delete [] El[i];
 delete [] El;
 El = new T*[b.Rows];
 for(i=0; i<b.Rows; i++)
 El[i] = new T[b.Cols];
 Rows = b.Rows;
 Cols = b.Cols;
 }
 for(i=0; i<Rows; i++)
 for(j=0; j<Cols; j++)
 El[i][j] = b.El[i][j];
 }
 return *this;
}

```

## 5. Kako će prevodilac reagirati prilikom prevođenja sljedećeg koda ?

a) 

```
class CRazred {
 void Draw() = 0;
};
CRazred object;
```

(5 bodova)

Prevodilac će javiti pogrešku budući da se pokušalo instancirati objekt apstraktnog klase (postoji čista virtualna funkcija unutar klase).

b) 

```
int Datum::Godina() const {
 this->god = 1990;
 return god;
}
```

(10 bodova)

Prevodilac će javiti grešku budući da se u `const`-članskoj funkciji pokušalo mijenjati stanje objekta za koji je funkcija pozvana (što nije dozvoljeno).